



A Hybrid Machine Learning and Deep Learning Stacking Ensemble Framework for Web Application Firewall Attack Detection

Submitted by
MD.SHAMINUR RASHID
ID: 222-35-1112
Department of Software Engineering
Daffodil International University

Supervised by
Dr. Rubaiyat Islam
Associate Professor
Department of Software Engineering
Daffodil International University

Thesis submitted in fulfillment of the requirements for the award of the degree
of Bachelor of Science, Department of Software Engineering (Major in Cyber
Security)

Spring 2026

APPROVAL



STUDENT'S DECLARATION

I hereby declare that the work in this thesis is my original work except for quotations and citations, which have been duly acknowledged. I also declare it has not been previously or concurrently submitted for any other degree at DAFFODIL INTERNATIONAL UNIVERSITY or any other institution.

Student's Signature

Name : MD. SHAMINUR RASHID
ID : 222-35-1112

Department of Software Engineering (Major in Cyber Security)
DAFFODIL INTERNATIONAL UNIVERSITY



SUPERVISOR'S DECLARATION

I hereby declare that I have checked this thesis and in my opinion, this thesis is adequate in terms of scope and quality for the award of the degree of Bachelor of Science.

Supervisor's Signature

Name : Dr. Rubaiyat Islam
Designation : Associate Professor

Department of Software Engineering
DAFFODIL INTERNATIONAL UNIVERSITY

THESIS DECLARATION LETTER

Librarian,
Daffodil International University
Daffodil Smart City, Ashulia.Dhaka,Bangladesh

Dear Sir,

CLASSIFICATION OF THESIS AS RESTRICTED

Please be informed that the following thesis is classified as RESTRICTED for a period of three (3) years from the date of this letter. The reasons for this classification are as listed below.

Author's Name: Md. Shaminur Rashid

Thesis Title

Reasons (i)

(ii)

(iii)

Thank you.

Yours faithfully,

(Supervisor's Signature) Date:
Stamp:

Note: This letter should be written by the supervisor and addressed to the Librarian, *Daffodil International University*, with a copy attached to the thesis.

ACKNOWLEDGEMENTS

First and foremost, I express my sincere gratitude to Almighty Allah for His infinite blessings, mercy, and guidance, which enabled me to complete this undergraduate thesis.

This research, titled "*A Hybrid Machine Learning and Deep Learning Stacking Ensemble Framework for Web Application Firewall Attack Detection*", was inspired by the growing need for intelligent cybersecurity solutions capable of detecting and mitigating modern web application attacks. The opportunity to contribute to this field has been both challenging and rewarding.

I would like to express my deepest appreciation to my thesis supervisor, Dr. Rubaiyat Islam, Associate Professor, Department of Software Engineering, Daffodil International University, for his invaluable guidance, constructive feedback, continuous encouragement, and unwavering support throughout every stage of this research. His expertise, insightful suggestions, and dedication to academic excellence played a vital role in shaping the direction and quality of this work.

I am also grateful to Dr. Imran Mahmud, Head of the Department of Software Engineering, Faculty of Science and Information Technology, Daffodil International University, along with all respected faculty members and staff of the department, for their cooperation, support, and encouragement during my undergraduate studies and research activities.

My heartfelt thanks go to my parents and family members for their unconditional love, patience, sacrifices, and continuous support throughout my academic journey. Their encouragement and belief in my abilities have been a constant source of motivation.

I would also like to thank my friends, classmates, and fellow students of Daffodil International University for their cooperation, valuable discussions, and moral support. Their companionship and encouragement made this journey more meaningful and enjoyable.

Finally, I acknowledge the researchers, organizations, and open-source communities whose datasets, tools, and resources contributed significantly to the completion of this study. Their efforts in advancing knowledge and promoting accessible research have been invaluable to this work. Without the support and contributions of all these individuals and organizations, the successful completion of this thesis would not have been possible.

ABSTRACT

Modern organizations rely heavily on web applications for financial processes, data handling, and digital services, making web-based injection attacks a serious threat. Static signature matching and the manual maintenance of a list of rules are intrinsic limitations of traditional Web Application Firewalls (WAFs) that leave an ongoing vulnerability to new attack variants, and can be bypassed using payload obfuscation methods. In this thesis, a machine learning-based WAF intrusion detection system is proposed that addresses four major categories of injection attacks, namely, Local File Inclusion (LFI), Cross-Site Scripting (XSS), SQL Injection (SQLi) and Operating System Command Injection (OSC), in a single multi-class classification approach.

The hybrid dataset was created by combining the CSIC 2010 and HTTPParams 2015 benchmark datasets with modern "real-time" HTTP attack traffic for all four types of attacks. To overcome the issue of class imbalance, the Synthetic Minority Over-sampling Technique (SMOTE) was employed with the training partition only (to avoid data leakage). The eleven candidate models were systematically tested on two parallel paths: six classical machine learning algorithms (XGBoost, Random Forest, LightGBM, CatBoost, Support Vector Machine (SVM) with RBF kernel and Logistic Regression); and five deep learning architectures (Deep Neural Network (MLP), TabNet, LSTM, CNN and Autoencoder). The best-performing models from the respective tracks were then separately hyper-optimised with RandomizedSearchCV and Optuna Bayesian optimisation, and their hyper-optimised configurations were combined together in a novel stacking ensemble, where the XGBoost/MLP models were used as base learners and the Logistic Regression model was used as the meta-learner, using out-of-fold predicted probabilities.

The experimental results show that XGBoost has the most stable classical performance with accuracy of 99.803% for XSS, 99.481% for SQL, 98.638% for OSC, and 93.842% for LFI. The DNN achieved the best deep learning results, especially for SQL injection, achieving an F1-Score of 0.9902. The proposed stacking ensemble had the overall highest accuracy of 96.09%, the highest Macro-F1-Score of 0.9592, and the highest ROC-AUC of 0.9765 for the merged multi-class dataset, and had the most balanced precision-recall profile among all the evaluated architectures. All models were assessed with five complementary metrics, namely: Accuracy, Macro-Precision, Macro-Recall, Macro-F1-Score, and ROC AUC, along with SHAP feature importance analysis, ROC curve evaluation, and confusion matrix examination. The outcomes demonstrate the effectiveness and suitability of the proposed stacking ensemble as a strong and production-ready architecture for multi-attack WAF intrusion detection.

Keywords: Web Application Firewall, Intrusion Detection, XGBoost, Deep Neural Network, Stacking Ensemble, SQL Injection, Cross-Site Scripting, Local File Inclusion, OS Command Injection, SMOTE, Optuna, SHAP

Table of Contents

APPROVAL	2
STUDENT'S DECLARATION	3
SUPERVISOR'S DECLARATION	4
THESIS DECLARATION LETTER	5
ACKNOWLEDGEMENTS	6
ABSTRACT	7
LIST OF TABLES	10
LIST OF FIGURES	11
LIST OF ABBREVIATIONS	12
CHAPTER 1 INTRODUCTION	13
1.1 Overview	13
1.2 Background	14
1.3 Problem Statement	16
1.4 Research Gap	17
1.5 Objectives	18
1.6 Motivation of the Study	18
1.7 Thesis Organization	19
1.8 Summary	20
CHAPTER 2 LITERATURE REVIEW	21
2.1 Introduction	21
2.2 Related Works	21
2.2.1 Web Application Security and Injection Vulnerabilities	21
2.2.2 Machine Learning-Based Web Application Firewalls	23
2.2.3 Classical Machine Learning Algorithms and Comparative Studies	24
2.2.4 Deep Learning Architectures for Intrusion Detection	25
2.2.5 Ensemble Learning and Stacking Generalization	26
2.2.6 Hyperparameter Optimization Techniques	27
2.2.7 Feature Engineering, Class Imbalance Handling, and Datasets	28
2.3 Conclusion	29
CHAPTER 3 RESEARCH METHODOLOGY	31
3.1 Overview	31
3.2 Data Preparation	32
3.2.1 Dataset Source and Data Collection	32
3.2.2 Hybrid Dataset Construction	33
3.2.3 Feature Harmonization and Feature Engineering	33
3.2.4 Missing Value (Null Value) Handling	34
3.2.5 Data Preprocessing	35
3.2.6 Data Splitting	35
3.2.7 Data Balancing Using SMOTE	35
3.3 Model Selection	36
3.3.1 Classical Machine Learning Models	36
3.3.2 Deep Learning Models	37
3.3.3 Selection Criteria for the Proposed Ensemble Model	37
3.4 Model Details	38
3.4.1 XGBoost	38
3.4.2 Random Forest	38
3.4.3 LightGBM	39
3.4.4 CatBoost	39
3.4.5 Support Vector Machine (RBF Kernel)	39
3.4.6 Logistic Regression	39
3.4.7 Deep Neural Network (MLP)	40

3.4.8	TabNet	40
3.4.9	LSTM (Long Short-Term Memory)	41
3.4.10	CNN (Convolutional Neural Network)	41
3.4.11	Autoencoder	41
3.4.12	Proposed Stacking Ensemble Architecture.....	41
3.4.13	Meta-Learner Design Using Logistic Regression.....	43
3.5	Model Training	43
3.5.1	Training Environment.....	43
3.5.2	Training Procedure	44
3.5.3	Hyperparameter Optimization	44
3.6	Evaluation Methods.....	45
3.6.1	Performance Metrics.....	45
3.6.2	Experimental Design	45
3.6.3	Comparative Performance Analysis	46
3.7	Summary.....	46
CHAPTER 4 RESULTS AND DISCUSSION		48
4.1	Overview	48
4.2	Result Analysis.....	48
4.2.1	Quantitative Performance Metrics.....	48
4.2.2	Metric-by-Metric Analysis	54
4.2.3	Relative Performance Analysis	55
4.2.4	Overfitting and Generalization Analysis	58
4.2.5	SHAP-Based Feature Importance Analysis (Proposed Model).....	58
4.2.6	ROC Curve Analysis	62
4.2.7	Confusion Matrix Analysis.....	64
4.3	Key Insights and Discussion.....	66
4.3.1	Classical versus Deep Learning: Performance Trade-offs	66
4.3.2	Challenges in Multi-Attack WAF Classification.....	67
4.3.3	Stacking Ensemble Architecture Benefits	67
4.4	Result Comparison	68
4.4.1	Ensemble Performance Across Individual Datasets	68
4.4.2	Comparative Analysis on the Merged Dataset	69
4.4.3	Computational Efficiency.....	70
4.4.4	Model Ranking Summary.....	72
4.5	Summary.....	73
CHAPTER 5 CONCLUSION		74
5.1	Overview	74
5.2	Summary of Research Findings.....	75
5.3	Research Contributions	76
5.4	Limitations of the Study	78
5.5	Future Work.....	79
5.6	Concluding Remarks	81
BIBLIOGRAPHY		83

LIST OF TABLES

Table 3.1 – Best XGBoost Hyperparameters per Sub-Dataset	38
Table 3.2 – Best MLP Hyperparameters from Optuna per Dataset	44
Table 4.1 – Classical ML Comparison: LFI Dataset.....	49
Table 4.2 – Classical ML Comparison: XSS Dataset	49
Table 4.3 – Classical ML Comparison: OSC Dataset.....	50
Table 4.4 – Classical ML Comparison: SQL Dataset	50
Table 4.5 – Deep Learning Comparison: LFI Dataset	52
Table 4.6 – Deep Learning Comparison: XSS Dataset.....	52
Table 4.7 – Deep Learning Comparison: OSC Dataset	53
Table 4.8 – Deep Learning Comparison: SQL Dataset.....	53
Table 4.9 – XGBoost Overfitting Analysis: Train–Test Performance Gap	55
Table 4.10 – Best XGBoost Hyperparameters per Dataset (MCW = min_child_weight)	56
Table 4.11 – MLP with Optuna: Performance Results Across Four Datasets	56
Table 4.12 – Best MLP Hyperparameters Selected by Optuna per Dataset	57
Table 4.13 – Stacking Ensemble (XGBoost + MLP → LR): Performance on Individual Datasets	68
Table 4.14 – Model Comparison on Merged Hybrid Dataset.....	69
Table 4.15 – Computational Efficiency Comparison of All Evaluated Models	71
Table 4.16 – Overall Model Ranking Summary	72

LIST OF FIGURES

Figure 3.1: Methodology Architecture.....	32
Figure 3.4.12: Proposed Stacking Ensemble Architecture.....	43
Figure 4.1: Classical ML F1-Score Comparison Across All Datasets.....	51
Figure 4.2: Deep Learning F1-Score Comparison Across All Datasets	54
Figure 4.3: Accuracy and F1-Score Side-by-Side: XGBoost vs DNN per Dataset	55
Figure 4.4: MLP Optuna Accuracy vs ROC-AUC Dual-Axis Chart.....	57
Figure 4.5: SHAP Feature Importance Bar Chart: Stacking Ensemble Model — LFI Dataset	59
Figure 4.6: SHAP Feature Importance Bar Chart: Stacking Ensemble— XSS Dataset	59
Figure 4.7: SHAP Feature Importance Bar Chart: Stacking Ensemble — SQL Dataset.....	60
Figure 4.8: SHAP Feature Importance Bar Chart: Stacking Ensemble— OSC Dataset.....	61
Figure 4.9: SHAP Feature Importance Bar Chart: Stacking Ensemble— Merged Hybrid Dataset.....	62
Figure 4.10: ROC Curves of the Proposed Stacking Ensemble on Individual Attack Datasets: (A) LFI, (B) XSS, (C) SQL, (D) OSC.....	63
Figure 4.11 – Multi-Class ROC Curves of the Proposed Stacking Ensemble	64
Figure 4.12: Normalized Confusion Matrices of the Proposed Stacking Ensemble on Individual Datasets: (A) LFI, (B) XSS, (C) SQL, (D) OSC.....	65
Figure 4.13: Confusion Matrix of the Proposed Stacking Ensemble: Merged Hybrid Dataset	66
Figure 4.14: Ensemble vs Individual Base Learners.....	69
Figure 4.15: Multi-Metric Comparison: Radar Chart on Merged Dataset.....	70

LIST OF ABBREVIATIONS

- LFI — Local File Inclusion
- XSS — Cross-Site Scripting
- SQLi / SQL — SQL Injection
- OSC — OS Command Injection (Operating System Command Injection)
- CSRF — Cross-Site Request Forgery
- XXE — XML External Entity (Injection)
- SSRF — Server-Side Request Forgery
- RFI — Remote File Inclusion
- MCW — min_child_weight (XGBoost hyperparameter)
- H1, H2, H3 — Hidden Layer 1, Hidden Layer 2, Hidden Layer 3
- TPE — Tree-structured Parzen Estimator
- OOF — Out-Of-Fold
- RSCV — RandomizedSearchCV

CHAPTER 1 INTRODUCTION

1.1 Overview

Modern society has undergone a major transformation in the digital landscape with the proliferation of web applications in all sectors, ranging from electronic commerce, online banking, healthcare information systems, government e-services, and social platforms. Today, individuals, organizations, and institutions share sensitive information, conduct financial transactions, and run critical operational processes primarily via web applications. This reliance on web-based services has also widened the attack surface for malicious actors who are constantly leveraging vulnerabilities in web application logic, input handling, and even the way they execute on servers to gain unauthorized access, exfiltrate data, and compromise the system.

To address this evolving threat landscape, Web Application Firewalls (WAFs) were invented as an intrusion detection and prevention (IDP) solution specifically focused on the Hypertext Transfer Protocol (HTTP) and the unique patterns of attacks targeting it. Conventionally, WAFs are based on a set of pre-fabricated rules and databases of known malicious request patterns. These methods will effectively block known attacks, but they can never evolve to address new, obfuscated, or zero-day attacks that don't match the defined rules, and they must be continuously manually updated as attack types change. The fixedness of rule-based systems has therefore spurred a huge research effort that focuses on machine learning-based WAFs that can learn to discriminate between labelled training data and generalize to new versions of attacks.

In this thesis, a complete machine learning approach for a WAF intrusion detection system is proposed to detect four major categories in web injection attacks, namely, Local File Inclusion (LFI), Cross-Site Scripting (XSS), SQL Injection, and Operating System Command Injection (OSC). The research systematically compares the performances of six classical Machine Learning algorithms and five Deep Learning architectures, optimizes the hyperparameters of the best-performing ones in each track, and proposes a new stacking ensemble of XGBoost and Deep Neural Network (MLP) base learners combined with a Logistic Regression meta-learner trained using out-of-fold predicted probabilities. The proposed system is tested on a hybrid dataset that is obtained by combining the real-time attack traffic with existing benchmark datasets, and class imbalance is addressed by using the Synthetic Minority Over-sampling Technique (SMOTE) only on the training partition.

1.2 Background

Requests and responses between browser and web server are structured in the way specified by the HTTP and HTTPS protocols. Each and every request to an HTTP server contains a specific structure: the URL path, the query parameters, the request headers, and the request body, which can all be manipulated by the attacker to force malicious content into the server-side processing pipeline. Web application attacks differ from network-layer attacks in that they are not a target of the infrastructure protocols, but rather of the application logic itself, exploiting the lack of proper input validation, error handling, or insecure configuration of the servers to execute commands, retrieve files, or modify database queries the application was never designed for. This attack vector is an application-layer attack, which means that it is allowed by the network firewalls and requires dedicated Web application security mechanisms for any Internet-facing application.

The OWASP Top 10 is a consensus document that represents the most serious security challenges to web applications, maintained by the Open Worldwide Application Security Project (OWASP). The latest version of injection vulnerabilities is ranked third out of the top ten most significant web application security weaknesses under category A03. This classification is indicative of the constant presence and impact of injection attacks, including some of the most harmful types such as SQL Injection, Cross-Site Scripting, OS Command Injection, and Local File Inclusion. Each of these attack types is based on a different functionality of the application, but they are all alike in this way: they all involve the injection of attacker-controlled data into an application context where it is interpreted as executable code, a query, or a reference to the file system instead of simply an input data element.

SQL Injection (SQLi) attacks occur when a user enters SQL keywords and operators into an input field, allowing an attacker to alter database query strings and, in some cases, make administrative calls on the database server, and in the worst-case scenario, get past authentication systems to retrieve, modify, or delete records in the database. Cross-site scripting (XSS) vulnerabilities allow an attacker to embed malicious JavaScript or HTML in web pages that are then rendered in legitimate users' browsers, allowing them to implement session hijacking, steal user credentials, and run malicious code on the user's machine. Operating System Command Injection (OSC) is an exploit of the functionality of a web application that passes user input to Operating System shell commands, and may give the attacker complete control of the underlying server Operating System. Local File Inclusion (LFI) attacks are those that exploit the file path parameters to point to files that are not in the normal server directory where they should be, which can give access to sensitive configuration files, authentication records, server logs, and, in some configurations, can allow remote code execution by poisoning the logs or including a malicious file in the uploads.

Historically, the basic technology used for monitoring and controlling network traffic against these threats has been Intrusion Detection Systems (IDS) and Intrusion Prevention Systems

(IPS). Web Application Firewalls (WAFs) are a special class of IDS/IPS applications designed to specifically target the semantics of HTTP traffic. There are two main detection techniques: signature-based detection, where incoming requests are matched against a database of known attack signatures, which is good for detecting known attacks but cannot detect new ones; and anomaly-based detection, where the statistical characteristics of normal traffic are modeled and deviations from this normal traffic are detected, potentially generating false positives for untracked attacks in this manner, but effectively for both known and unknown attacks.

Traditional rule-based WAFs, like MoD Security, which is built on regular expression patterns from the OWASP Core Rule Set, require security engineers to manually craft and maintain regular expression patterns that define attack payloads. These systems are multi-year-old and in place, but they have two fundamental drawbacks. First, attacks can evade signature-based detection by using encoding methods, payload obfuscation, and evading signatures at the protocol level, while maintaining the semantics of the malicious request and avoiding signature matches. Second, the rule maintenance burden is large and persistent – with new attack variants and exploitation techniques, there must be new rules to keep detection coverage [3,4]. Due to these restrictions, it has inspired a lot of research on machine learning-based WAFs that can learn discriminative features automatically from labelled examples and update attack patterns by retraining.

In the case of machine learning for WAF intrusion detection, the problem of HTTP request classification is considered as a supervised binary/multi-class classification problem. A request is encoded as a feature vector based on the engineered attributes of its structural elements such as the URL path, request body, request headers, and a trained classifier classifies the request to the benign or attack class. Common features in HTTP traffic include lexical and syntactic properties of request fields (such as the number of times attack-indicative keywords appear, the number of times special characters associated with payloads of injection attacks appear, and the length of the structure of request components). To decrease the number of features to boost the generalization of the classifier, statistical feature selection techniques like chi-square, pointwise mutual information, and mutual information are used to identify the subset of features with the maximum discriminative power between attack and benign classes [6,18].

Tree-based ensemble algorithms, especially Gradient Boosting algorithms such as XGBoost, LightGBM, and CatBoost, have proved to perform well across various structured tabular classification tasks, among the classical machine learning algorithms. Introduced by Chen and Guestrin [11], XGBoost is a very efficient and overfitting-resistant implementation of regularized gradient boosting with a second-order Taylor expansion of the loss function and explicit L1 and L2 regularization terms to control the complexity of the trees. Neural network architectures like Multi-Layer Perceptrons, attention-based models like TabNet, and sequential models like LSTMs provide complementary advantages of non-linear hierarchical representations of abstracted features from raw features and/or features with minimal engineering, and have demonstrated potential for use in intrusion detection, where feature interactions are complex, and the number of features is large enough to train neural networks.

Some ensemble methods, such as "stacking", where the meta-learner is trained on the prediction made by each base model, have been found to enhance the robustness of the classification process compared to any single model [8] and especially when the base models have complementary inductive biases.

1.3 Problem Statement

Although much research has been done on machine learning-based Web Application Firewalls (WAFs), deploying a practically implementable, comprehensive, and robust ML-based WAF for detection of real-world multi-attacks remains a technically and methodologically challenging problem. The basic problem comes from the fact that there are several requirements that have to be met at the same time. The production WAF should have high recall of all types of attacks with as few misses as possible and high precision, which means that as few users as possible should be blocked from using the legitimate service. Because attackers continually change their tactics to evade detection, it must be able to generalize to attack payloads that are structurally different from those in the training data. It should be able to run with low enough inference latency to be able to be deployed in the live HTTP request processing pipeline in a way that doesn't cause noticeable delays. And it has to be tested on a representative set of data that mirrors real HTTP traffic – both with modern payloads for attacks and real, representative benign traffic patterns.

New WAF technologies, based on machine learning, usually support just one or two of these needs separately. Many published studies have tested their models on a single attack type - most commonly XSS or SQL injection alone - and haven't determined if their approach can be generalized to the entire range of injection attacks a production WAF should be designed to handle. Still others use only benchmark data like CSIC 2010, which was based on web traffic patterns over 10 years old and fail to take into account the characteristics of the structure of today's attack payloads. In many attack scenarios, the number of attack samples may be significantly lower or higher than the number of benign samples, which is not addressed by many real-world attack datasets, and thus these datasets can lead to models that are over-trained on the majority class and under-trained on the minority class that may represent the most operationally critical detection target. In addition, many of the previous works use a specific classification algorithm without systematically testing other architectures, and it is not certain that the algorithm they use is the correct one for the detection task or just the first one that was tested.

To overcome these limitations, in this research, a comprehensive ML-based WAF IDS is built to detect all four major categories of injection attacks at once; principled class rebalancing is done using SMOTE, and a systematic comparison is done of eleven candidate models belonging to classical and deep learning paradigms, followed by the development of a stacking ensemble architecture of the best-performing models of both paradigms. To ensure the models

are trained and tested on both modern and historically documented attack patterns, the hybrid dataset is used for training and evaluation, consisting of a mix of contemporary real-time attack traffic and established benchmark data.

1.4 Research Gap

Through a literature review, gaps in the literature are identified that this research will fill. The most common restriction in the existing literature is a limited attack surface. Many of the published machine learning WAF studies are limited to detecting two types of attack: XSS and SQL injection, the two most commonly included in publicly available datasets[5,6,7]. In fact, there are fewer studies that include both LFI and OSC injection and even fewer that include and consider all four types of attacks in a single multi-class study. This limited scope implies that published metrics for the current models might not reflect the actual results of using them in a real-world WAF setting where all four attack types occur at the same time.

Another key gap is the lack of use of ensemble learning and systematic hyperparameter optimization in the research on WAF IDS. Most published works test only single classifiers with a particular set of hyperparameters, typically the default ones or manually selected, and do not use principled search procedures like RandomizedSearchCV or Bayesian optimization frameworks to find the hyperparameters that maximize generalization performance for the particular detection problem. Though suitable for combining complementary tree-based and neural network learners, stacking ensemble architectures have been largely ignored as a WAF classification approach, though they have proven to yield significant performance improvements over individual models in related machine learning competitions and applied research settings [8]. In WAF research, there is no exploitation of ensemble approaches, which is a missed opportunity in order to enhance the robustness of detection, especially in ambiguous cases close to the boundaries of classes, in which the decisions of individual classifiers differ.

The third one is a systematic comparison of classical machine learning and deep learning architectures in a single controlled experimental setup for WAF ID. There are two basic manners for class classification research: classical methods and deep learning methods; most of the studies have chosen one of the two methods from the beginning, but there is still a lack of empirical research on what method is more suitable for the characteristics of the attack classification task of the HTTP protocol. Comparative performance of ensembles of trees against neural architectures in the case of engineered tabular HTTP features — for which classical models often perform as well as or better than their deep learning counterparts — is not often characterized with the same kind of experimental rigour to inform deployment decisions. Moreover, there is a lack of uniformity in the literature when dealing with the class imbalance issue with injection attack datasets, where some studies will oversample the whole dataset before splitting, and thus, data leakage can occur, resulting in inflated performance metrics, and other studies will not rebalance at all, producing classifiers that are biased toward

the majority class [9,10]. This research tackles all three gaps by performing a controlled multi-model comparison with adequate train-test separation of SMOTE application and evaluation with respect to all four major injection attack categories, and by applying a stacking ensemble of the top models of both paradigms.

1.5 Objectives

The main goal of this research is to achieve the following objectives:

- To create a new hybrid dataset, combining real-time HTTP attack traffic from the current era with previous, well-known benchmark datasets (CSIC 2010 and HTTPParams 2015), including all four injection attack types (LFI, XSS, SQL Injection and OSC), and use SMOTE to deal with the class imbalance, but only in the training part of the datasets.
- To systematically compare six classical machine learning models (XGBoost, Random Forest, LightGBM, CatBoost, SVM with RBF kernel and Logistic Regression) and five deep learning models (MLP/DNN, TabNet, LSTM, CNN and Autoencoder) for all four individual attack datasets and the combined hybrid dataset.
- To optimize the best-performing models from each track (RandomizedSearchCV with 200 iterations and 10-fold cross-validation for XGBoost, and Optuna Bayesian optimization with 100 trials for the MLP) in a dedicated manner, and describe the train-test generalization gap to ensure robustness.
- Stacking ensemble architecture: To design and evaluate a stacking ensemble architecture where the base learners are tuned XGBoost and MLP, and the meta-learner is Logistic Regression, by avoiding data leakage between the base learner and meta-learner training process using out-of-fold predictions.
- To design a model with optimal architecture for production-level deployment of WAF, by presenting a detailed performance analysis including Accuracy, Macro-Precision, Macro-Recall, Macro-F1-Score, and ROC-AUC, along with SHAP-based feature importance analysis, ROC curve evaluation, and confusion matrix examination.

1.6 Motivation of the Study

This research is motivated by the confluence of two realities: the rapid proliferation of attacks via the Web and the inability of static rules-based security mechanisms to keep up with the attacks. Today attacks on Web applications represent a significant and rapidly increasing percentage of the total number of cybersecurity incidents taking place all over the world.

Injection attacks are the third most critical vulnerability category according to OWASP and they can be especially harmful because they allow attackers to break into the basic data handling logic of web applications, which will give them access to private databases, to run arbitrary commands on the server, or even to view the contents of the private file system by using requests that appear to be legitimate to the network-layer firewall. The financial, reputational and regulatory ramifications of a successful injection attack into a banking, healthcare or government web application are potentially serious and can include the loss of money, the cost of notifying affected data owners, regulatory penalties, and a gradual loss of user trust.

In this context, the drawbacks of conventional WAFs become critically important. An up-to-date signature-based WAF will not scan the attack, but will pass it straight on to the web server. If an attacker URL-encoded or double-encoded a SQL injection attack string, a rule can be matched only against the plain-text version of the string may fail to detect the attack. These aren't hypothetical weaknesses, but real weaknesses that are exploited every day, and the fact that the rate of new attack variants is outpacing the rate of rule set updates means there will always be a window of opportunity for them to be exploited — and that's where the machine learning-based approaches can make a fundamental difference. If the underlying feature representation captures the semantic characteristics of the class of attacks, on a rich and diverse dataset of both attack and benign traffic, a model could be generalized to structurally novel payloads that were not present in the training set. Such generalization, along with the fact that the models can be retrained as new data emerges, makes the use of ML-based WAFs a promising and real-world relevant research field.

The specific motivation for the stacking ensemble proposed in this thesis is that through empirical observation, it is found that the tree-based gradient boosting architecture and the neural network architecture show complementary strengths with regards to HTTP classification tasks. In particular, XGBoost is capable of making very precise and high-confidence classifications on attack payloads, where its feature activations are very discriminative, whereas MLP based neural networks give better recall for attack types that have more diffuse feature signals, such as LFI patterns that share some overlap with legitimate file path references. These strengths can be combined in a principled ensemble framework to create WAF classifiers that are both more precise and more complete than either approach alone, directly applicable to production WAF systems' operational needs.

1.7 Thesis Organization

This thesis consists of five chapters, each corresponding to a different part of the research. Chapter 1 has given the research domain, background of the Web Application Security and Web Application Firewall (WAF) technology, problem statement, research gap, and the objectives and motivations for the research. The relevant literature is extensively

reviewed in Chapter 2, and various previous works are analyzed, including machine learning-based IDS, web application firewall design, feature engineering for analyzing web application traffic, classical classification methods, deep learning classification methods, ensemble methods, and hyperparameter optimization. This chapter demonstrates how the proposed research fits in the existing body of work and also highlights the contribution of this research. Chapter 3 outlines and explains the research methodology, the process for acquiring the datasets and building the hybrid dataset, the process for harmonizing and engineering the features, the procedure for rebalancing the classes using SMOTE, the choice and architectural design of all eleven candidate models, the hyperparameter optimization procedure for XGBoost and MLP, the stacking ensemble architecture and out-of-fold training procedure, and the evaluation framework along with the five performance metrics and the experimental design. The entire experimental outcomes including comparison between classical and deep learning models on all four attack datasets, hyperparameter tuning analysis, SHAP feature importance analysis, ROC curve analysis, confusion matrix analysis and comparison of proposed stacking ensemble models with base learners on merged hybrid attack datasets are presented and analysed in Chapter 4. The thesis is finalized in Chapter 5, where the main outcomes of the thesis are summarized, implications of the results for WAF deployment are addressed, the limitations of the thesis are discussed, and future directions are proposed, such as extending the thesis to encompass more attack categories, spatially-aware sequential feature extraction, and adversarial robustness evaluation.

1.8 Summary

The research context, problem definition, and objectives have been set for the machine learning-based Web Application Firewall intrusion detection system for four critical categories of injection attacks in this chapter. WAFs based on signatures are known to be insufficient in tackling new attack patterns that can be seen in today's fast-growing web applications, and the need to detect these attacks adaptively and using learning-based approaches is very evident. The three specific contributions that this thesis makes to the field are based on the research gaps identified in the literature reviewed; namely, the narrow attack coverage, the lack of ensemble architectures, and systematic hyperparameter optimization, as well as the inconsistent treatment of class imbalance. To overcome these limitations, the proposed stacking ensemble method that utilizes the complementary capabilities of XGBoost and MLP using Logistic Regression as the meta-learner is presented and tested within a comprehensive experimental framework that considers all four types of attacks as well as proper data handling procedures, and performance is analyzed in both the individual and merged multi-class settings. The next chapters describe the technical development and experimental validation of this approach in detail.

CHAPTER 2 LITERATURE REVIEW

2.1 Introduction

The necessary literature is analyzed in detail to place the research of this thesis in the wider context of the field, in terms of web application security, machine learning for intrusion detection, deep learning architectures, ensemble classification, and hyperparameter optimisation. This chapter reviews existing research in a systematic way with respect to seven thematic areas that together outline the theoretical, empirical, and methodological underpinnings for the proposed stacking ensemble web application firewall (WAF). The surveyed publications range from baseline research on injection vulnerability features to the classical WAF design, classical machine learning and deep learning methods for classification of HTTP attacks, to recent work in ensemble stacking, hyperparameter search using Bayes, and dataset construction for WAF research.

This chapter follows a hierarchical format, corresponding to the tiered nature of the research problem. One of the first steps to understanding why a specific feature engineering strategy or model architecture is more suitable for detecting a certain type of attack is to understand how the attack can be carried out. The three stages- rule-based WAF, anomaly-based WAF, and machine learning-based WAF- show the direction that encourages the choice of approach used in this thesis. Algorithms in Sections 2.2.3 to 2.2.5 are relevant to the 11 candidate models tested during the experimental study, while the data handling and optimization techniques outlined in Sections 2.2.6 and 2.2.7 support the data pre-processing and training pipeline presented in Chapter 3.

The work reviewed in this chapter is analyzed in terms of both contributions and limitations and methodological gaps found in each work, which helps in motivating the design decisions of the proposed system. In Section 2.3, the chapter is summarized with a synthesis of the literature which clarifies the particular research questions that this thesis attempts to answer.

2.2 Related Works

2.2.1 Web Application Security and Injection Vulnerabilities

Since the advent of dynamic, server-side Web applications in the late 1990s, Web application security has been a key focus of cybersecurity research. The request structure of the HTTP protocol is used for all communications between the client and the web server and has multiple injection points: the URL path, query string parameters, request header, and the body of the request. If the server-side application fails to properly validate and sanitise all user-

provided data before handling it, it can be manipulated by an attacker. Injection vulnerabilities have always been in one of the top three spots of the OWASP Top 10, a consensus-based list of the most dangerous web application security risks, which has been continuously maintained by the Open Worldwide Application Security Project (OWASP) since the first edition, due to their ubiquity across the Web and the dramatic impact that exploiting them can have[1,15]. In an early comparative study, Ghanbari et al. discovered that organizations that went with OWASP Top 10 guidelines as the foundation for their security postures were able to provide a more solid protection against injection attacks than those that only used vendor-specific rule sets, which points to the real-world benefits of having standardized vulnerability taxonomies for WAF design [15].

The four attack types considered in this thesis use injection to exploit an application functionality, but each use the same injection mechanism, namely injecting attacker-controlled data into a context in which the server-side processes it as executable code, queries, or file system references, not as simple input. SQL Injection attacks inject SQL statements into parameter values of database queries, allowing an attacker to manipulate the database records and to circumvent authentication. Cross-Site Scripting attacks are attacks in which malicious JavaScript or HTML is inserted into the content of a page that is displayed in the victim's browser, allowing client-side execution of malicious scripts. OS Command Injection is a vulnerability in Web applications that provides the ability to execute shell commands, which may result in full server access. LFI are vulnerabilities in which the attacker can modify a file path parameter to point to files outside of the server's intended directory. Knowing the syntactic and lexical properties of each attack type, namely the presence or absence of certain special characters, operators, and keywords in their payloads, is essential to the feature engineering approach that was taken in this work: using statistical methods such as chi-square testing, pointwise mutual information, and mutual information, we identified those features that contain highest discriminative power for each attack category [8].

The drawbacks of signature- and rule-based WAFs in the context of these injection threats are well known in the security research literature. Luptk explored WAF bypass techniques and how attackers can use URL encoding, case variations, the insertion of comments and other payload obfuscation techniques to evade signature-based detection, while still maintaining the semantic maliciousness of the request and bypassing pattern-matching rules [3].

Shipulin also added that the biggest issue with signature-based IDS is that the signatures are inherently reactive, meaning that they can only be used to detect attack patterns that have already been documented and defined as signatures, leaving zero-day variants and new obfuscation techniques permanently beyond their reach. Stankovic and Simic suggested a comprehensive strategy for web application security: using WAF, input sanitisation and output encoding at the application level; since no individual security layer is enough, and using WAF together with secure development practices is optimal[17]. These results, taken together, demonstrate that the structural limitations of rule-based WAFs cannot be solved by refining their rules and that adaptive, data-driven approaches are essential to provide strong protection

against emerging injection attacks.

2.2.2 Machine Learning-Based Web Application Firewalls

It was the bypass vulnerabilities of signature-based systems that gave rise to the use of machine learning in designing Web Application Firewalls. Isiker and Sogukpinar presented a WAF that utilizes natural language processing methods along with Support Vector Machine (SVM) with kernel linear. They modeled the HTTP request content using both word n-gram and character n-gram representations and tested them with CSIC 2010 benchmark data set [5]. Their character n-gram model obtained an overall accuracy of 99.53% which is better than previous works on the same dataset. The study proved that using the HTTP request as a textual sequence and feature extraction based on the NLP approach can be extremely discriminant for attack detection, but the only use of a benchmark dataset and the lack of a real-time evaluation in a real web environment restrict the generalizability of the performance figures obtained in the study to deployment scenarios.

Shaheed and Kurdy proposed a WAF model that makes use of features from three elements of HTTP requests: URL path, request payload, and request headers. A man-in-the-middle proxy was deployed to process the incoming requests and several features such as request length, ratio of allowed characters to special characters and an attack severity weight were calculated and then utilized to train four classifiers: Naïve Bayes, Logistic Regression, Decision Tree and SVM, using both train-test-split and cross-validation to reduce overfitting [6]. The models were tested on CSIC 2010 dataset, HTTPParams 2015 dataset, and hybrid of both with real-world server logs with as high as 98.8% accuracy on hybrid case. The width of the feature engineering of this work indirectly determined the feature construction methodology used in this research. However, the sample sets for OSC and LFI attack types were limited, resulting in only small sample sizes for the reported performance for those attack types; and no ensemble combination of the algorithms evaluated was explored.

Román-Gallego et al. created an AI-powered WAF able to protect against all four types of injection attacks: SQLi, XSS, OSC injection, and path traversal (LFI), by leveraging a synthetic data set of more than 100,000 requests generated from publicly available injection payload repositories [7]. The SVM outperformed the other classical algorithms (Naïve Bayes, KNN, SVM and Logistic Regression) in overall accuracy, with the F1-Scores being 0.97 in command injection, 0.97 in path traversal, 1.0 in XSS and 1.0 in SQL injection. This study is one of the few that provides a comprehensive assessment on all 4 attack types studied in this thesis, but lacks the inclusion of a real evaluation dataset comprising of HTTP traffic logs, deep learning architectures, assessment of ensemble methods, and systematic evaluation of hyperparameters. These restrictions are a primary impetus in the design of the experimental framework of this thesis.

Durmuskaya and Bayrakli created a hybrid dataset by integrating the HTTP attack traffic

collected by them via automated attacks on intentionally vulnerable websites such as DVWA, bWAPP, etc., with the CSIC 2010 and HTTPParams 2015 benchmark data sets, where the payloads were executed with the help of Burp Suite [8]. This hybrid dataset was used for the evaluation of five classical ML algorithms (KNN, Logistic Regression, Naïve Bayes, SVM and Decision Tree) and the best one (Decision Tree) was implemented in a live WAF environment using the mitmproxy tool. The accuracy of real-time testing using Burp Suite is 93.27% and F1-Score is 93.13%, which is one of the few published studies that used real-time testing of WAF using ML instead of offline simulation. This work offers a hybrid dataset and the framework for feature engineering which proves to be the experimental base of the present work, carrying significant extension to the hybrid dataset through deep learning architectures, ensemble stacking and systematic hyperparameter optimization.

2.2.3 Classical Machine Learning Algorithms and Comparative Studies

Various classical ML algorithms have been tested on ID datasets to determine which algorithm best suits to deploy in WAF systems. In the CICIDS2017 network intrusion dataset, Guven et al. compared the accuracy performance of the classifiers of Random Forest, Decision Tree, Logistic Regression and Naïve Bayes and concluded that Random Forest outperforms other classifiers and linear models with 99.94% accuracy for multi-class network traffic classification [10]. Ismail et al. conducted a more comprehensive evaluation of eight supervised ML algorithms, namely KNN, Naïve Bayes, Logistic Regression, SVM, Decision Tree, Random Forest, AdaBoost, and ANN, on the CSIC HTTP dataset, which revealed that the highest accuracy of 89% was obtained by KNN and Random Forest with an AUC of 94% [18]. Naïve Bayes was seen as the most efficient in terms of computation even if it performed less than the rest, and the results of the present study indicated that Naïve Bayes has always lagged behind the tree-based ensembles. However, both studies use the same dataset and do not consider the performance per attack type, as well as address class imbalance, reducing the value of these studies to be a straight forward comparison for multi-attack WAF design.

Harish Kumar et al. compared classical algorithms with deep learning models such as CNN and LSTM along with tree-based models for the detection of XSS attack and found that the gradient boosting variants performed better than other classical methods and CNN and LSTM showed competitive performance, which is, however, more expensive in terms of computation [20]. As part of the evaluation of the different algorithms, Kascheev and Olenchikova compared SVM, Logistic Regression, Decision Tree and Naïve Bayes specifically for XSS detection, and stated that Decision Tree obtained the highest recall at 93.70%, precision of 99.19% and overall accuracy of 98.81%, arguing that it is impossible to evaluate the effectiveness of any classification algorithm using a single measure, a stance that is also followed in this thesis [21].

The family of gradient boosting algorithms — XGBoost, LightGBM, and CatBoost — is the most powerful and extensively tested family of classical ML algorithms for structured tabular classification problems. To address the limitations of gradient boosting, Chen and Guestrin

proposed XGBoost, a regularized gradient boosting framework that optimizes a differentiable loss function with explicit L1 and L2 regularization on the complexity of the trees, which works well across a wide range of classification benchmarks and is also efficient in computation due to using column block structures for parallel tree learning. Ke et al. created LightGBM, an implementation of gradient boosting using histogram-based learning and leaf-wise tree growth, which can save memory and training time while keeping the accuracy competitive to XGBoost [24]. Prokhorenkova et al. proposed CatBoost which solved the above-mentioned prediction shift problem in gradient boosting with ordered boosting and applied symmetric oblivious decision trees which gives extra regularization against overfitting [25]. The underlying bagging ensemble method used, called Breiman's Random Forest, builds many decorrelated decision trees that classify the class label based on the maximum voting, which is highly resistant to overfitting [26]. As mentioned in this thesis, the performance of these four very different tree-based ensembles was remarkably similar on most of these datasets, suggesting that when a low-dimensional feature representation is highly engineered, any well-designed tree ensemble can achieve a similar performance ceiling.

2.2.4 Deep Learning Architectures for Intrusion Detection

Despite its ability to automatically learn hierarchical features from raw and/or minimally engineered representations, deep learning architectures have been recently adopted for intrusion detection and web application security applications. Kassem built a host-based intrusion detection system (HIDS) based on the techniques of text mining and the DOC2VEC representation of the content of the HTTP request to classify malicious URLs in a dataset of 6,000 malicious URLs for directory traversal, XSS and SQL injection vulnerabilities [23]. The highest overall accuracy rate of 90.67% was obtained with the MLP, and SVM had the best accuracy rate of 95.41% for path traversal detection. This study stands out in that it shows that neural network methods can be used to achieve performance comparable to classical methods on WAF classification problems using proper text representations, but it lacks the ability to cover OSC injection due to the small size of the dataset, limiting its generalisability.

Dawadi, Adhikari and Srivastava suggested a deep learning based WAF that included LSTM networks trained using ISCX, CSIC 2010 and CICDDoS datasets, which resulted in 97.57% accuracy in detecting DDoS and 89.34% for SQL injection and XSS. Though this study showed that LSTM can be used for WAF intrusion detection, and validates the deep learning approaches in more dynamic traffic than just benchmark based evaluations, the accuracy on SQL and XSS combined at 89.34% is less than other classical tree based approaches for those specific attacks in this thesis, indicating that the sequential modelling capability of LSTM is not a significant advantage for raw token sequences of the HTTP request transformed into engineered scalar counts of features. The Habibi and Surantha study of XSS detection using Naïve Bayes, KNN and SVM with n-gram features further proved that text-sequence-based representations can be very effective for some classes of attacks, but it was XSS only and didn't investigate deep learning architectures [22].

The vanishing gradient problem in recurrent neural networks was solved by introducing gated memory cells in the Long Short-Term Memory (LSTM) network, which allows modelling long-range sequential dependencies in time-series data [27]. LSTM has proven to be effective in natural language processing, network flow analysis, and log-based intrusion detection, but the results in this thesis indicate that the benefit of using LSTM over feedforward architectures is not as great when the input representation is in the form of counts of features rather than raw sequences of tokens. Arik and Pfister proposed an attention-based deep learning architecture, called TabNet, for tabular data, which is able to select a sparse instance-wise subset of the most informative features by using sequential attention at each decision step [28]. The interpretability of TabNet using attention weights and its performance on structured datasets make it an obvious choice to classify WAF attacks – the know-how of driving features in each prediction can be useful for security analysts during operation. The evaluation in this thesis validates that TabNet can perform competitively with MLP in most attack datasets, and it provides extra model interpretability, with MLP having a better performance on SQL injection, which demands a deeper feature combination.

2.2.5 Ensemble Learning and Stacking Generalization

The combination of several classifiers to achieve accurate and robust results is known as ensemble methods and has a sound theoretical and empirical background in the literature of machine learning. The stacked generalization approach trains a meta-learner on the predictions of a number of base classifiers, as opposed to the raw training data, allowing the meta-learner to learn the strengths and weaknesses of each base learner and to combine their predictions to reduce the errors of the individual models [29]. The rationale for stacking is that if each of the base learners erred on a different subset of the input space, then a sufficiently rich meta-learner would be able to learn to trust the base learner when it is in correct and to discount it when it is not. This property makes stacking particularly well suited to the WAF classification problem, since there are complementary error patterns between the two types of models: Gradient boosted models tend to have higher precision, that is, they tend to make precise classifications with high confidence when the data points are clearly discriminative; neural networks on the other hand tend to have higher recall on borderline cases, which have smoother decision boundaries.

Ensemble methods have been used for several related tasks for cybersecurity classification. In the work of Guven et al., Random Forest ensemble exhibited a significant improvement over the individual classifiers and linear classifiers for the problem of Network Intrusion Classification [10]. They found AdaBoost, a boosting ensemble, to be competitive with the other 7 algorithms they tested for web attack detection [18]. In the literature, however, stacking ensembles of heterogeneous base learners (coming from different algorithmic families) such as a gradient boosting model and a deep neural network have not been systematically studied for WAF intrusion detection. Liu, Li and Chen suggested to use search-based to generate WAF

test cases by using differential evolution to traverse the semantic space of SQL injection payloads, showing that the combination of complementary optimization strategies can reveal vulnerabilities that cannot be found by using a single-strategy approach [33]. Then, Li, Yang, and Visser generalized this to DANUOYI – an evolutionary multitask injection testing framework for generating test cases for multiple types of injections by multi-task learning and cross-lingual translation [34]. Though the focus for these works is on WAF evaluation and not on WAF construction, these works reiterate the fact that collaboration of complementary approaches gives a wider coverage than using a single approach, the same principle that is used in this thesis for stacking ensembles.

2.2.6 Hyperparameter Optimization Techniques

Machine learning models can be highly sensitive to the values of their hyperparameters — architectural and training parameters that aren't learned from data; they are fixed before the model is trained. Current methods of hyperparameter selection are either unreliable or too computationally expensive when dealing with high dimensional hyperparameter spaces such as those found in deep learning, and require extensive manual tuning depending on the domain's experience. Randomized search (first proposed as a principled alternative to grid search) uniformly samples the space of hyperparameter configurations at random from a continuous distribution over the space of hyperparameters, and has been demonstrated to perform as well as grid search, or even better, for fewer hyperparameter configurations to evaluate: often, the performance of a model will depend on only a small number of hyperparameters, so that a dense grid search over the non-influential ones wastes compute budget [12]. Sklearn's `RandomizedSearchCV` implementation is employed in this thesis for XGBoost tuning and utilizes the random search idea within a cross-validation framework, and tests each hyperparameter combination on several held-out folds, giving a statistically stable performance estimate.

Bayesian hyperparameter optimization avoids random search by creating a probabilistic model of the objective function: the performance of the cross-validated model as seen from the hyperparameter configuration. This model can then be used to select the next configuration to test. Optuna is a next-generation hyperparameter optimization framework that implements the Tree-structured Parzen Estimator (TPE) algorithm, which models the hyperparameter space as a tree-structured probability distribution and picks the next configuration to evaluate by maximizing the ratio of the probability of achieving a good objective value to the probability of achieving a bad one [12]. The APIs and pruning mechanisms of Optuna enable defining more complex hyperparameter spaces with conditions and early termination of unpromising trials, greatly minimizing the overall compute budget needed for optimization. For the neural network hyperparameter optimization to classify WAFs, Optuna's ability to efficiently traverse a joint distribution of the learning rate, dropout rate, weight decay, and hidden layer size is significantly superior to manual or grid search methods in determining an MLP configuration with good generalization over a range of different attack datasets examined in this research.

2.2.7 Feature Engineering, Class Imbalance Handling, and Datasets

To detect intrusions reliably, any machine learning model must rely on a quality/representative dataset and the informativeness of the feature representation. In [6] Shaheed and Kurdy found that the frequencies of attack-indicative keywords and special characters in the original URL path and request body, the discriminative power of each of these frequencies measured by chi-square, pointwise mutual information, and mutual information, provide a compact, yet general and informative feature representation for the classification of HTTP requests, which can be efficiently computed in real-time and across different attack types. This feature engineering approach, which underlies the feature representation used in this thesis, captures the lexical content of attack payloads by mapping keywords to their frequency (counts) in the payload, and the syntactic structure by mapping special characters to their frequency (counts) in the payload without deep parsing of the request body or understanding its application-layer context.

Class imbalance is a ubiquitous problem in intrusion detection datasets, in which the proportion of benign requests to attack requests may be orders of magnitude in favor of benign; or in which the number of samples for a particular type of attack may be significantly lower than the number of benign samples. When a classifier is trained on an imbalanced dataset without any of the proper measures, the result is a model that is biased toward the majority class, with an artificially high accuracy, but low recall of the minority attack class. Chawla et al. have proposed Synthetic Minority Over-sampling Technique (SMOTE) which creates synthetic samples for the minority class by interpolating between some of the minority class instances in the feature space, thus augmenting the minority class, but not by simply duplicating minority class samples [9]. One of the key conditions mentioned in the SMOTE literature and generally followed in this thesis is that oversampling should only be performed on the training partition after the train-test split, and that test sets should not contain any synthetic data, and that performance metrics should be based on the generalization ability to real-class test distributions.

The two most popular benchmarks used in the WAF research community have been the two datasets described above. Giménez, Villegas and Marañón created the labelled HTTP dataset, CSIC 2010, by automatically generating HTTP requests against a simulated ecommerce web application, normal and anomalous requests are generated comprising SQL injection, XSS, buffer overflow and parameter tampering attacks [13]. Although CSIC 2010 is the most often referenced WAF benchmark, there are several drawbacks; first, it was built in 2010 in the simulation of traffic and not real production web apps, and second, it does not contain the new attack variants that have come in the last ten years since its construction. Some of these temporal limitations are partially overcome by Morzeux's HTTPParams 2015 dataset, which contains a newer set of HTTP parameters, but without OSC injection payloads [14] either. The currency and coverage of these datasets were identified as a limitation by Durmuskaya and

Bayrakli as they developed a hybrid dataset that joins together CSIC 2010 and HTTPParams 2015 with real-time attack traffic created by automated simulations against modern vulnerable Web applications from active security repositories, such as SecLists, PayloadsAllTheThings, and payloadbox [8,19], to provide labelled HTTP logs of all four injection attack categories with modern payload variants. This dataset is a combination of both datasets, which is the experimental data foundation for the present research.

In the area of supervised ML, Shar, Briand, and Tan used Random Forest and Logistic Regression classifiers with cross-validation and cross-project validation procedures to detect LFI and found recall rates of 81% and 76%, respectively, for LFI detection [31]. To evaluate the feasibility of the attack detection model for LFI, Tao and Meng created an autoencoder-based model with a round-robin fashion, and tested it on real campus network traffic to obtain a classification accuracy of up to 95.3% for detecting file inclusion attacks, demonstrating that unsupervised reconstruction-based methods have reasonable detection rates for LFI when they are trained on representative traffic in the target deployment environment [32]. However, the autoencoder approach investigated in the present thesis, which was trained on the hybrid engineered-feature data set and not the raw URL text, consistently failed to reach a recall value of 15% or higher in all four attack categories, suggesting that the autoencoder-based anomaly detection paradigm is fundamentally not applicable to data sets based on aggregated feature counts, as it was impossible to train the autoencoder to classify high-frequency keyword co-occurrences as anomalous without using a sequential or contextual representation of the data set.

2.3 Conclusion

The reviewed literature shows a clear trend in the research of web application firewalls that starts with the static signature based systems, which have a documented set of bypass attacks, followed by the single algorithm ML-based WAF systems, which show promising results for the individual attacks, but are not yet general enough, and the initiation of comprehensive multi-attack ML-based WAF systems that integrate engineered HTTP features with classical classifiers. From the reviewed works, the conclusion is that on the features of structured HTTP requests, the machine learning models typically perform well in terms of classification accuracy, and in particular the gradient-boosted tree ensembles. On the other hand, deep learning models provide complementary strengths in terms of recall on ambiguous border cases which are not reflected in any single classical model.

Although some advancements have been reported in the literature, there are still a few gaps that need to be addressed. No prior work has ever attempted such a comprehensive comparison across the entire spectrum of both traditional and deep learning model families, across all four categories of injection attacks, all within a controlled framework, in a modern hybrid dataset. There remains an important question of whether stacking ensemble architectures of tree-based

gradient boosting and neural network base learners offers measurably better performance than a single well-tuned model for WAF intrusion detection, which has not been addressed. Combining RandomizedSearchCV with gradient boosting classifiers in the same experimental framework, as well as applying Bayesian hyperparameter optimization to neural network WAF classifiers, have not been investigated systematically. Although oversampling is common practice for the treatment of class imbalance in the general intrusion detection community, some papers introduced oversampling techniques prior to the splitting operation, which introduces leakage, and others have not performed any rebalancing whatsoever in the case of WAFs.

All these gaps are filled in this thesis in a single experimental setup. This is the most comprehensive comparison of different models in the WAF intrusion detection literature to date, which includes six classical ML algorithms and five deep learning architectures, evaluated on four attack datasets and a merged multi-class setup. The proposed stacking ensemble, consisting of a tuned XGBoost base learner and an Optuna-optimized MLP base learner, is a novel addition to the design of WAF ensembles. By using SMOTE only in the training partition, and by systematically measuring train-test generalization gaps of all tuned models, the reported performance numbers give honest and reliable estimates of performance when deployed in production.

CHAPTER 3 RESEARCH METHODOLOGY

3.1 Overview

This chapter provides a detailed account of the methodology used to create and test a machine learning-based Web Application Firewall (WAF) Intrusion Detection System (IDS). The methodology covers all phases of the research process, such as data acquisition and hybrid data construction, feature harmonization, feature engineering, data preprocessing, handling class imbalance using the Synthetic Minority Over-sampling Technique (SMOTE), model selection and architecture design, training procedures with hyperparameter optimization, and a multi-metric evaluation framework.

The main goal of this research is to create a high-performing, robust WAF that is able to detect four main categories of web injection attacks, which include SQL Injection (SQLi), Cross-Site Scripting (XSS), Local File Inclusion (LFI), and Operating System Command Injection (OSC). For this purpose, a detailed comparative study of classical machine learning models and deep learning architectures was performed, and a novel stacked ensemble model was proposed by combining XGBoost and a Multilayer Perceptron (MLP) as base learners with Logistic Regression as a meta-learner.

The rest of this chapter provides a description of the data preparation process (Section 3.2), model selection criteria Section 3.3, the architectural details of all candidate models and the proposed ensemble Section 3.4, training procedures (Section 3.5), and evaluation procedures Section 3.6. The data preparation process is described in Section 3.2, the model selection criteria in Section 3.3, the architectural details of all candidate models and the proposed ensemble in Section 3.4, training procedures in Section 3.5, and evaluation procedures in Section 3.6, all of which are followed. The process of data preparation is outlined in Section 3.2, the process of model selection in Section 3.3, the architectural details of all candidate models and the proposed ensemble in Section 3.4, training procedures in Section 3.5, and evaluation procedures in Section 3.6 are all followed. The pipeline of the study is shown in Figure 3.1

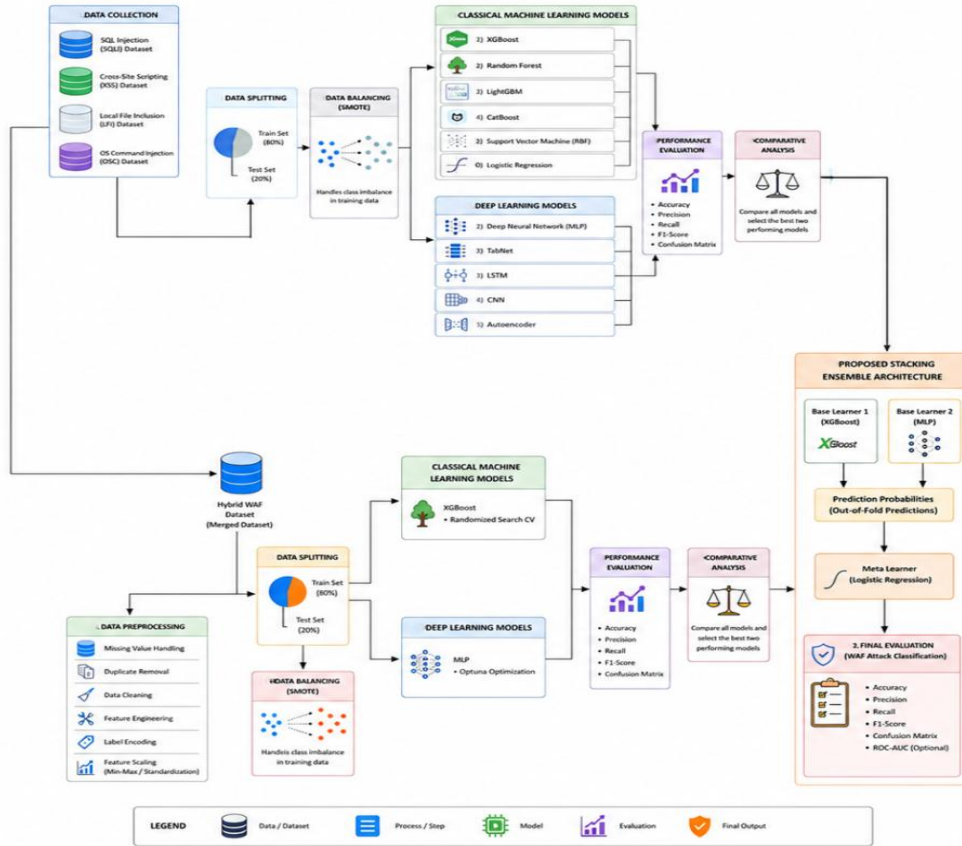


Figure 3.1: Methodology Architecture

3.2 Data Preparation

3.2.1 Dataset Source and Data Collection

The information for this study was obtained from Figshare (id: 26583526), which is available through the DOI: <https://doi.org/10.6084/m9.figshare.26583526.v1>. This dataset was first used in the machine learning-based WAF study of Durmuskaya and Bayrakli (2025) and is made up of HTTP request logs covering both malicious and benign web traffic.

The data collection was performed by using Burp Suite to conduct automated attack simulations on intentionally vulnerable web applications such as demo.testfire.net, testphp.vulnweb.com, bWAPP, and juice-shop.herokuapp.com. The attack payloads were taken from popular security feeds like SecLists, PayloadsAllTheThings, Offensive-Payloads, and payloadbox. In addition to the real-time attack logs, two widely used benchmark datasets, i.e., the CSIC 2010 HTTP dataset and the HTTPParams 2015 dataset, are integrated.

In this research, four sub-datasets of attack-specific data were used. The SQL Injection (SQLi)

dataset contains 20,000 malicious requests and 20,000 benign ones. The total number of requests analyzed is the sum of the malicious requests and the benign requests that are displayed in this dataset. The total number of requests analyzed is the number of malicious requests plus the number of benign requests shown in this dataset. The Local File Inclusion (LFI) dataset consists of 41,000 malicious requests and 20,000 benign requests. The Local File Inclusion (LFI) dataset contains 41,000 malicious requests and 20,000 benign requests. OS Command Injection (OSC) dataset: 21,000 malicious requests and 20,000 benign requests. The pre-processing was carried out independently for each sub-dataset and combined to form the hybrid WAF dataset for the comparative and ensemble models.

3.2.2 Hybrid Dataset Construction

A Hybrid Dataset was created by combining the four attack-specific sub-datasets together for the purpose of multi-class intrusion detection and for full model evaluation. The building process was systematic, going through the following pipeline: The analysis of features was done for the four sub-datasets to find common and attack-specific features.

All sub-datasets were given the same common features to preserve compatibility when merging. A feature union approach was employed to combine attack-specific features that had discriminative power in specific attack classes together into a single schema, where missing values were filled in for other attack classes in the record.

A new classification of the target labels was created, where each attack type was assigned a class label: 0 for benign traffic; 1 for SQL Injection; 2 for XSS; 3 for LFI; 4 for OSC Injection. All sub-datasets were merged row-wise to form the final merged dataset. To guarantee label correctness and structural integrity, data consistency validation was carried out after the merger.

The composite dataset consists of records for each of the four attack classes and benign samples, which creates a realistic and comprehensive training environment for the WAF models. This hybrid construction method is suitable for the live HTTP traffic real-world operational needs of a WAF, which must detect multiple attack types simultaneously.

3.2.3 Feature Harmonization and Feature Engineering

Before merging the attack datasets, a harmonization step of the features was performed, which was an important preprocessing step. The following operations were done: Column Name Standardization: All columns in the datasets were standardized with the same naming scheme. Data Type Unification: Type inconsistencies in feature values were removed by converting all the feature values to compatible numerical data types. Duplicate and Redundant Attribute Removal: Redundant features and duplicate columns were identified and removed.

The features used were engineered using the statistical extraction method given by Durmuskaya and Bayrakli (2025). The engineered features are syntactic and lexical properties of the HTTP requests, and they contain: Bad-word count: Counts of attack-indicative keywords (such as ‘whoami’, ‘passwd’, ‘alert’, ‘script’, ‘exec’, ‘union’, ‘select’) in the HTTP request path, content, and header fields. Special characters count: Number of special characters in the text, like single quotes, angle brackets, semicolons, slashes, ampersands, hyphens, and other special characters. Three complementary statistics, the Chi-square test, Pointwise Mutual Information (PMI), and Mutual Information (MI) have been used to validate feature significance. This multi-measure technique makes sure that only features with good and consistent discriminative power are kept. The chi-square statistic is used to test if a feature is dependent on the target class based on the comparison of observations and expectations.

$$\chi^2 = \sum (O_i - E_i)^2 / E_i \quad \chi^2 = \sum E_i (O_i - E_i)^2$$

where O_i represents the observed frequency and E_i the expected frequency if the events were independent. The Pointwise Mutual Information (PMI) is a measure of how well a feature xxx is associated with a class yyy, given as:

$$PMI(x, y) = \log \frac{P(x, y)}{P(x)P(y)} \quad PMI(x, y) = \log \frac{P(x, y)}{P(x)P(y)} \quad PMI(x, y) = \log P(x)P(y)P(x, y)$$

The joint probability of feature and class co-occurrence is $P(x, y)$, and the probabilities of features and of classes are $P(x)$ and $P(y)$ respectively. The Mutual Information (MI) is the sum of PMI over all the possible outcomes of the feature and the target variable, and it provides the overall dependency between these two variables.

$$I(X; Y) = \sum_{x \in X} \sum_{y \in Y} P(x, y) \log \frac{P(x, y)}{P(x)P(y)} \quad I(X; Y) = \sum_{x \in X} \sum_{y \in Y} P(x, y) \log \frac{P(x, y)}{P(x)P(y)} \quad I(X; Y) = \sum_{x \in X} \sum_{y \in Y} P(x, y) \log P(x)P(y)P(x, y)$$

where XXX denotes the feature space, and YYY denotes the class labels.

3.2.4 Missing Value (Null Value) Handling

In the integration of the datasets, missing values were created because the features of the attacks were missing in the other sub-dataset. To overcome this problem, the constant-value imputation method was used, which means that the entries missing from the data were replaced with zero. This is suitable for request-based features and frequency-derived features (when there is no token or event, there is no value).

To minimize noise and enhance data quality, features with more than one-half of the total

missing data values (based on the combined data from all merged features) were excluded. Data consistency checks were done after imputation to prevent any nulls in the training and test sets. All preprocessing stages were done in a leakage-free way, where transformations were only applied to the training partition, and then applied to the validation and test sets.

3.2.5 Data Preprocessing

After addressing missing values, the data underwent several preprocessing steps before being used for model training:

Feature Scaling: StandardScaler was used to normalize all the numerical features. This step is especially critical for Support Vector Machines (SVM), Logistic Regression, and MLP models, due to the fact that they can be sensitive to the scaling of features. The transformation is computed as:

$$z = (x - \mu) / \sigma$$

where x is the raw feature value, μ is the mean of the feature over the training samples, and σ is the standard deviation of the feature over the training samples.

Label Encoding: Integer values were used for the target class labels to make them compatible with all machine learning frameworks.

Feature Validation: A last consistency check was done to ensure that all feature values were finite, non-null, and within expected ranges.

3.2.6 Data Splitting

All datasets were divided using an 80-20 stratified train-test split. Each of the attack sub-datasets as well as the merged hybrid dataset was split into 80% training and 20% testing data. To ensure the class distribution of the training and test sets was representative of the class distribution of the original data set, stratification was imposed, especially considering the intrinsic class imbalance in intrusion detection sets.

All the experiments employed a fixed random seed (`random_state = 42`) for splitting the data to ensure data reproducibility across runs. After splitting, only the training partition was used with the SMOTE (oversampling) technique, and no synthetic data from the training partition have been added to the test partition. This separation is very important in order to obtain unbiased performance estimates when evaluated.

3.2.7 Data Balancing Using SMOTE

Some sub-datasets of the individual attacks had class imbalance, where there were more

malicious than benign samples in certain sub-datasets. For the sub-dataset LFI, without any resampling, the distribution of classes was 22,716 malicious and 8,461 benign. Machine learning leads to an imbalance of classes, resulting in a bias towards the majority class for the machine, which negatively affects the minority class's recall. To solve this problem, the training partitions of each attack data sets were enhanced by using the Synthetic Minority Over-sampling Technique (SMOTE). SMOTE makes synthetic instances of the minority class from existing instances in the minority class in the feature space through interpolation. The synthetic sample x_{new} is created for each minority sample x_i according to the following procedure:

$$x_{new} = x_i + \lambda * (x_k - x_i)$$

x_k is a randomly chosen k-nearest neighbor of x_i from the same class, and λ is a random scalar uniformly chosen between 0 and 1. In this research, the value k was given a value of 5 (default SMOTE setting). The SMOTE method was used to balance the LFI training partition, resulting in the size of the training partition increasing from 24,941 to 36,344 samples, and the class distribution became 18,172 samples per class. The other sub-datasets were similarly balanced, as necessary. To avoid the synthetic data affecting the test of models on real data, SMOTE was used only on the training set.

3.3 Model Selection

3.3.1 Classical Machine Learning Models

To provide a complete performance benchmarking of various algorithmic paradigms, a set of six baseline classifiers were selected from classical machine learning models:

- XGBoost (Extreme Gradient Boosting): Selected due to its success in tabular data, the properties of its regularization, and its empirical results for structured classification tasks, was selected.
- Random Forest was chosen since it is an ensemble-based approach to avoid overfitting by creating a random training set, and its ability to handle high-dimensional feature space.
- LightGBM: This is a gradient boosting algorithm that is optimized for speed and memory usage, especially for large datasets, and was included.
- CatBoost: Chosen for its use of categorical features and its symmetric tree structure used for gradient boosting.
- Support Vector Machine with RBF Kernel (SVM): Selected due to its robustness and theoretical support in the field of classification in high-dimensional spaces.

- Logistic Regression: Linear baseline used and then used as the meta-learner for the stacking ensemble.

3.3.2 Deep Learning Models

To capture the non-linear and sequential patterns in the data of HTTP traffic, five deep learning architectures were tested:

- Deep Neural Network / MLP (Multilayer Perceptron): Researched for Tabular Classification and has been successfully applied with proper regularization and hyperparameter tuning.
- TabNet: Included as an attention-based deep learning architecture specifically designed for tabular data, offering interpretability through sequential attention mechanisms.
- LSTM (Long Short-Term Memory): Good for modeling sequential dependencies, which could be useful for temporal patterns in request sequences.
- CNN (Convolutional Neural Network): It was added to assess the feasibility of spatial feature extraction, which is used for the feature vectors of the structured HTTP requests.
- Autoencoder: The autoencoder was benchmarked as an unsupervised baseline to detect anomalies: requests with high reconstruction error were marked as anomalies while normal traffic was reconstructed.

3.3.3 Selection Criteria for the Proposed Ensemble Model

All the candidate models were then compared, and a stacking ensemble was designed as follows:

The XGBoost was chosen as a base learner due to its high performance across all four individual sub-datasets concerning the accuracy, F1-score, and ROC-AUC of the classical machine learning models, which indicates its superior performance. The second base learner was chosen as an MLP (Deep Neural Network) due to its excellent nonlinear learning capability over all the evaluated deep learning architectures, especially in the merged hybrid dataset. Logistic Regression was chosen as the meta-learner because of its simplicity, low computational complexity, and it has been found to be effective as a combination of the prediction probabilities of various diverse base learners in a stacking ensemble framework. The integration of the outputs of the gradient boosting (XGBoost) and the neural network-based learning (MLP) is done in a principled fashion by using the Logistic Regression meta-learner, and this combination takes advantage of the complementary strengths of the two.

3.4 Model Details

3.4.1 XGBoost

The Extreme Gradient Boosting (XGBoost) model is an ensemble tree model that creates a series of decision trees, with each new tree fixing the errors of the previous ones. The objective function that is optimized during training is made up of a differentiable loss term and a regularization term:

$$L(\theta) = \sum_i [l(y_i, \hat{y}_i)] + \sum_k [\Omega(f_k)]$$

where l denotes the training loss function (log-loss for binary classification), y_i is the true class label, $\hat{y}_i$ is the predicted class label, f_k is the k -th tree and $\Omega(f_k) = \gamma T + (1/2) \lambda \|w\|^2$ is the regularization term that penalizes the complexity of trees based on their number of leaves T and the norm of the leaf weights $\|w\|^2$.

For this research, XGBoost was fine-tuned with the RandomizedSearchCV method and 200 iterations with 10-fold cross-validation. The hyperparameter search space included $n_estimators$ in $\{200, 300, 400, 500\}$, max_depth in $\{4, 5, 6, 7, 8\}$, $learning_rate$ in $\{0.01, 0.05, 0.10, 0.15\}$, $subsample$ in $\{0.7, 0.8, 0.9, 1.0\}$, $colsample_bytree$ in $\{0.7, 0.8, 0.9, 1.0\}$, γ in $\{0, 0.1, 0.2\}$, and min_child_weight in $\{1, 3, 5\}$. The optimization criterion for the cross-validation was the macro F1-Score.

Table 3.1 – Best XGBoost Hyperparameters per Sub-Dataset

Hyperparameter	LFI	XSS	OSC	SQL	Merged
$n_estimators$	300	300	400	500	600
max_depth	5	4	4	6	10
$learning_rate$	0.10	0.05	0.05	0.10	0.05
$subsample$	0.9	0.8	1.0	0.9	0.9
$colsample_bytree$	0.9	1.0	0.8	0.8	0.7
γ	0.1	0.2	0.2	0.2	0.3
min_child_weight	3	1	1	1	1

3.4.2 Random Forest

Random Forest is an ensemble method of bagging that builds several independent

decision trees from a bagging of the training set, with each tree taking a random subset of features at each split. The final classification is the majority vote of all the trees. As a strong baseline, Random Forest was also tested using default hyperparameters from the scikit-learn package and is known to be resistant to overfitting.

3.4.3 LightGBM

LightGBM (Light Gradient Boosting Machine) is a gradient boosting framework that employs histogram-based learning and leaf-wise tree growth, offering significantly faster training than traditional gradient boosting on large datasets. LightGBM was evaluated with default hyperparameters as a comparative gradient boosting baseline alongside XGBoost and CatBoost.

3.4.4 CatBoost

CatBoost is one of the gradient boosting algorithms that applies ordered boosting to avoid prediction shift and ordered target statistics from categorical feature encoding. In particular, it uses symmetric (oblivious) decision trees that offer advantages in terms of regularization. Finally, CatBoost was also evaluated as an additional comparison point against gradient boosting using default hyperparameters.

3.4.5 Support Vector Machine (RBF Kernel)

In the newly transformed feature space, Support Vector Machines (SVMs) try to find the maximum-margin hyperplane that separates the classes. For example, an SVM with the Radial Basis Function (RBF) kernel will automatically exploit an infinite-dimensional feature space in which to classify non-linearly separable data. It is defined by the kernel function:

$$K(x_i, x_j) = \exp(-\gamma * ||x_i - x_j||^2)$$

Here, gamma defines the radius of influence for each training sample. As a classical non-parametric baseline, the SVM was assessed with default scikit-learn parameters ($C = 1.0$, $\gamma = 'scale'$).

3.4.6 Logistic Regression

Building a new area of expertise in a growing field such as quantum cybersecurity. Training for researchers in Hybrid Quantum-Classical systems. Stability has now been proven in other fields, beyond the WAF realm, e.g., to fraud detection and network security. Logistic Regression is a model where the log-odds of class membership is modeled for each instance

using the sigmoid (logistic) function applied to a linear combination of input features. The predicted probability for binary classification is:

$$P(y = 1 | x) = 1 / (1 + \exp(-(w^T * x + b)))$$

Where w is the weight vector, x is the feature vector, and b is a bias term. In other words, people denote this as a kind of probabilistic algorithm for multi-class classification, and it generalizes this formulation with the softmax function. Due to the computational efficiency and interpretability in combining base learner probability outputs, Logistic Regression was incorporated both as a standalone classical baseline and as the meta-learner of the stacking ensemble architecture.

3.4.7 Deep Neural Network (MLP)

The Deep Neural Network (DNN), which is a Multilayer Perceptron (MLP), includes one input layer, a number of fully connected hidden layers with nonlinear activation functions, and a single softmax layer for multi-class classification. For layer l , an equation for forward propagation is:

$$h^{(l)} = \text{activation}(W^{(l)} * h^{(l-1)} + b^{(l)})$$

The output layer l' is formed by the output layer l , the weight matrix $W^{(l)}$, and the bias vector $b^{(l)}$. All of the hidden layers used the ReLU activation function. Hyperparameter optimization was carried out by implementing a Bayesian optimization framework called Optuna. The optimal parameters were the learning rate, dropout rate, weight decay, and the number of neurons in three hidden layers. The Adam optimizer was used for backpropagation with early stopping to prevent overfitting. The optimal hyperparameters found for the merged data were: learning rate = 0.00128; dropout = 0.174; weight decay = 2.21e-6; and hidden layer sizes of 128, 64, and 128 neurons.

3.4.8 TabNet

TabNet is a tabular neural network architecture that focuses on the attention mechanism. It uses sequential attention at every decision step to choose a sparse set of the most relevant features for each sample, delivering high performance and instance-level interpretability. As a deep learning method, TabNet was tested as an alternative to MLP, especially because it natively works with the feature space of an HTTP request without the need for much feature engineering.

3.4.9 LSTM (Long Short-Term Memory)

A Recurrent Neural Network (RNN) with the gated memory unit is called a Long Short-Term Memory (LSTM) network. The LSTM units have three gates, namely: Input, Forget, and Output gates, which control the flow of information:

$$\begin{aligned}f_t &= \sigma(W_f * [h_{t-1}, x_t] + b_f) \\i_t &= \sigma(W_i * [h_{t-1}, x_t] + b_i) \\C_t &= f_t * C_{t-1} + i_t * \tanh(W_C * [h_{t-1}, x_t] + b_C) \\o_t &= \sigma(W_o * [h_{t-1}, x_t] + b_o) \\h_t &= o_t * \tanh(C_t)\end{aligned}$$

Here f_t , i_t , o_t are the forget, input, and output gate activations, C_t is the cell state, and h_t is the hidden state. Sequential modelling is evaluated to see if there are advantages when modelling HTTP request features.

3.4.10 CNN (Convolutional Neural Network)

Convolutional Neural Networks use learned filters to learn local feature patterns by convolutional operations. In 1D structured data like HTTP feature vectors, 1D convolutional layers are able to learn local co-occurrence patterns of neighboring features. Convolution and Pooling layers are followed by fully connected layers for final classification. CNN was added to test the effectiveness of local feature extraction for intrusion detection.

3.4.11 Autoencoder

Autoencoder — An unsupervised neural network architecture to reconstruct its input via a compressed bottleneck representation. At inference time, samples with reconstruction error above a threshold (determined by the model) are labelled as anomalous. A baseline using the Autoencoder was added into the unsupervised anomaly detection group in order to unambiguously evaluate whether or not reconstruction based approaches can be made possible for WAF intrusion detection against WAF traffic in a largely data-poor domain.

3.4.12 Proposed Stacking Ensemble Architecture

The proposed model is a two-level stacking ensemble of XGBoost and MLP for better classification accuracy when compared to the single XGBoost and MLP models. The architecture consists of:

The base layer includes two heterogeneous base learners:

Base Layer (Level-0 Learners): A gradient boosting tree ensemble with good performance on structured tabular features: XGBoost.

Multi-layer perceptron (Deep neural network): Non-linear feature abstraction and complementary representation learning, provided by a multi-layer perceptron.

Meta Layer (Level-1 Learner): Logistic Regression takes as input the out-of-fold predicted class probabilities of both base learners and gives out the final class label. Stacking ensemble training procedure is used to avoid data leakage, and it is done based on out-of-fold (OOF) k-fold approach. For K folds; The training set is segmented into K equally sized parts. Both models are trained on the K-1 folds that are not folded over and make predictions on the k-th fold to obtain OOF probability vectors P_{XGB} and P_{MLP} . The entire OOF predictions (shapes: $n_{train} \times n_{classes}$ for P_{XGB} and P_{MLP} , respectively) are then tallied side-by-side to create the meta-feature matrix $M = [P_{XGB} \mid P_{MLP}]$. The true labels are y_{train} , and M is trained with a Logistic Regression meta-learner. To make inferences, both base models are used to predict the probability of the class for each test sample, and the vectors of the class probabilities of the base models are fed to the meta-learner, which finally predicts the class label.

Final ensemble forecast:

$$y_{hat} = LR(XGBoost(x), MLP(x))$$

The Logistic Regression meta-learner (LR) and the vectors of class probabilities generated by the two base models ($XGBoost(x)$ and $MLP(x)$) are denoted here by the letters `xgboost` and `mlp`. The diagram of the proposed stacking ensemble architecture is shown below in Figure 3.4.12:

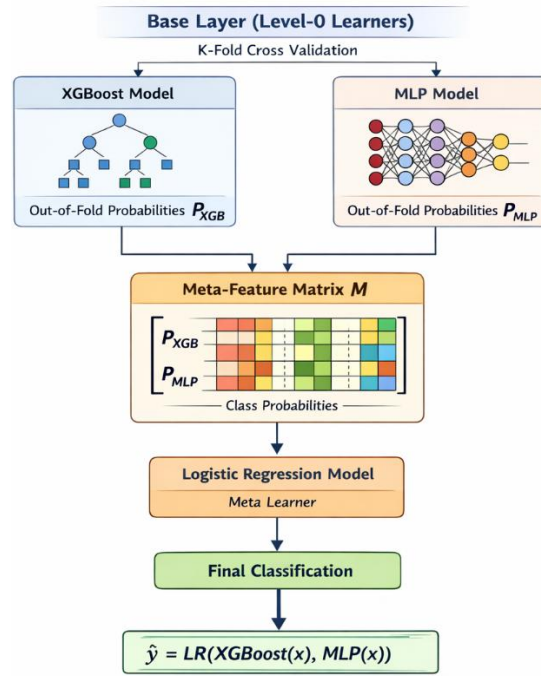


Figure 3.4.12: Proposed Stacking Ensemble Architecture

3.4.13 Meta-Learner Design Using Logistic Regression

The following reasons were why Logistic Regression was chosen as the meta-learner. First, it offers a principled probabilistic component for aggregation of the probability outputs of base learners, where it takes these as confidence scores. Second, it provides low latency during inference, making it an important feature for a real-time WAF deployment where latency is crucial. Third, L2 regularization (ridge) in Logistic Regression helps prevent overfitting of the OOF meta-features by the meta-learner. Fourth, the learned meta-learner coefficients offer an interpretable line of information on the relative contribution of each of the base learners in the final classification decision.

3.5 Model Training

3.5.1 Training Environment

Google Colab was used to train all models with access to NVIDIA Tesla T4/V100 GPUs having 16 GB of VRAM. Mainly, we used scikit-learn for classical machine learning models, PyTorch 2.0+ with CUDA 11.8 for the deep learning models, and XGBoost 1.7+ for gradient boosting models. To optimise learning rates and free up GPU memory, mixed precision training (FP16) and Automated Mixed Precision (AMP) were used for deep learning models.

3.5.2 Training Procedure

Common training that are used in all models is Normal distribution (mean = 0, standard deviation = 0.02) was used to initialize the weights for the layers of a neural network. Clipping gradients with a maximum norm of 1.0 was used during training deep learning models to make sure they converged smoothly. Model checkpointing was implemented every 10 epochs, and the best validation loss checkpoint was saved. Deep learning models were allowed to stop early after 10 epochs of validation loss. In the case of XGBoost-SMOTE, training was done using the SMOTE-resampled training set, and testing was done using the original unmodified test set. This procedure allows for the overfitting analysis (the difference between train and test accuracy) to be an honest reflection of the generalization capability, and not the result of synthetic oversampling. Optuna models were used to optimize the MLP models, and 100 trials were run per dataset by training the MLPs for at most 50 epochs with the Adam optimizer and cosine annealing learning rate scheduling.

3.5.3 Hyperparameter Optimization

In this study, two optimization strategies for hyperparameters were used:

RandomizedSearchCV (XGBoost): 200 randomly sampled hyperparameter combinations were tested with 10-fold cross-validation, and the macro F1-Score was used as the optimization metric. Randomized search is an effective alternative to exhaustive grid search and random sampling.

Optuna Bayesian Optimization (for MLP): Optuna uses a Tree-structured Parzen Estimator (TPE) algorithm to model the objective function space and perform efficient exploration of hyperparameter space. The following hyperparameters of the MLP were optimized using Optuna:

The following learning rate is sampled log-uniformly in [1e-5, 1e-2].

The uniform sampling is in the range [0.1, 0.5] for the dropout rate.

Weight decay (L2 regularization): log-uniform sampling of [1e-8, 1e-3]

Hidden layer sizes (H1, H2, H3): categorical choices from {32, 64, 128, 256, 512}

Table 3.2 – Best MLP Hyperparameters from Optuna per Dataset

Hyperparameter	LFI	XSS	SQL	OSC	Merged
Learning Rate	0.00314	0.00354	0.00314	0.00314	0.00128
Dropout	0.185	0.219	0.185	0.185	0.174
Weight Decay	5.34e-7	2.87e-7	5.34e-7	5.34e-7	2.21e-6

H1 (neurons)	512	128	512	512	128
H2 (neurons)	256	64	256	256	64
H3 (neurons)	128	32	128	128	128

3.6 Evaluation Methods

3.6.1 Performance Metrics

Five standard metrics were used to evaluate all models on a held-out test set:

Accuracy: The ratio of correctly classified instances for all samples:

$$Accuracy = (TP + TN) / (TP + TN + FP + FN)$$

Precision (Macro): The ratio of the average true positive predictions to the average of all positive predictions for each class, averaged:

$$Precision = TP / (TP + FP)$$

Recall (Macro): The average ratio of true positive predictions to all actual positive samples, averaged over each class:

$$Recall = TP / (TP + FN)$$

F1-Score (Macro): The harmonic mean of precision and recall to achieve a balanced measure:

$$F1 = 2 * (Precision * Recall) / (Precision + Recall)$$

ROC-AUC: Receiver Operating Characteristic area under the curve; metric of the model's classification power over all classification levels. For multi-class, a macro-averaged ROC-AUC was calculated.

Macro-averaging was chosen as the aggregation strategy because it gives equal weight to all attack classes, which is significant in a WAF scenario where rare attack types are equally critical to detect as frequent ones. Confusion matrices were also examined to determine the patterns of misclassifications between the classes.

3.6.2 Experimental Design

The experimental evaluation consisted of two levels:

Phase 1 – Individual Attack Dataset Experiments: The six classical ML models and five deep learning architectures used were trained and evaluated individually on each of the four individual attack sub-datasets (LFI, XSS, SQL, OSC). This led to a full performance comparison of all model/dataset combinations.

Phase 2 – Merged Hybrid Dataset Experiments: The best model performances of both classical models (XGBoost) and deep learning models (MLP with Optuna) were then tested on the combined hybrid dataset. Additionally, the stacking ensemble model was tested on the combined dataset, which was used to evaluate the benefit of ensemble integration.

To quantify the overfitting risk for the XGBoost hyperparameter tuning, a train-test accuracy gap analysis was carried out. A gap threshold of less than 0.04 was considered good generalization, 0.04-0.08 as moderate overfitting, and more than 0.08 as high overfitting. The gaps for all evaluated XGBoost models are within the low overfitting range.

3.6.3 Comparative Performance Analysis

The comparative analysis was done in the following three dimensions:

Classical vs. Classical: For each sub-dataset, XGBoost, Random Forest, LightGBM, CatBoost, SVM, and Logistic Regression were compared and evaluated on the performance of the best classical ML approach. To find the best-performing deep learning architecture among five different models (MLP, TabNet, LSTM, CNN, Autoencoder), the models were compared on each sub-dataset. To quantify the performance gains of integrating the models of the proposed stacking ensemble (XGBoost + MLP + Logistic Regression meta-learner), the models were tested on the combined dataset against individual XGBoost and MLP models.

3.7 Summary

This chapter described in detail the research methodology adopted for developing and evaluating a machine learning-based WAF intrusion detection system. The methodology includes a comprehensive multi-stage pipeline starting with the acquisition of a real-world HTTP attack dataset from Figshare, where four independent attack sub-datasets, corresponding to four different attacks (SQL Injection attack, XSS attack, LFI attack, OSC Injection attack) are joined to form a unified hybrid dataset.

Systematic feature harmonization, missing value imputation, and feature scaling were used to ensure data quality. A new approach to class imbalance, SMOTE, was used only on the training partitions to avoid spoiling the evaluation process. Overall, eleven candidate models from six classical ML algorithms and five deep learning architectures were tested on a consistent 80/20 split with a 20% test set of the stratified dataset.

After comparing the classifiers, the authors chose XGBoost and MLP as base classifiers to form the proposed stacking ensemble model, using Logistic Regression as the meta-classifier. To get both base models to perform at their optimal level before they were used in the ensemble, hyperparameter optimization was performed using RandomizedSearchCV for XGBoost and Optuna Bayesian optimization for MLP.

The metrics used were five complementary metrics (Accuracy, Precision, Recall, F1-Score, ROC-AUC) with macro-averaging, which ensures that the evaluation is done on an equal footing between all attack classes. The results of the experiments and detailed performance comparisons will be given in the next chapter.

CHAPTER 4 RESULTS AND DISCUSSION

4.1 Overview

This chapter is a comprehensive evaluation of all the machine learning and deep learning models proposed for web application firewalls (WAF) intrusion detection. The results of the experiments are presented and organized to facilitate a systematic comparison of the classical machine learning algorithms with the deep learning architectures, on four individual attack sets (Local File Inclusion (LFI), Cross-Site Scripting (XSS), Operating System Command Injection (OSC), SQL Injection) and a finally merged hybrid dataset comprising all four attack sets as a multi-class classification problem. All models were trained and evaluated within the same 80/20 stratified train-test split, with SMOTE only used to handle the class imbalance during the training phase to avoid introducing synthetic samples into the test set to impact the evaluation results.

The same study is presented in two parallel tracks. For the classical track, 6 algorithms are evaluated: XGBoost, Random Forest, LightGBM, CatBoost, Support Vector Machine (SVM) with RBF kernel, Logistic Regression. Five architectures are assessed in the deep learning track: Deep Neural Network (DNN/MLP), TabNet, LSTM, CNN, and Autoencoder. For further comparison, the best model in each track (XGBoost and MLP) was trained with dedicated hyperparameter search, using RandomizedSearchCV and Optuna Bayesian search, respectively. The predicted probabilities of the out-of-fold classification for each of these tuned base learners were then stacked in the proposed stacking ensemble, with logistic regression trained as a meta-learner.

Five complementary metrics are used to compare all models: Accuracy, Micro-Precision, Micro-Recall, Micro-F1-Score, and ROC-AUC. To have the same weight for each attack class, the macro-average was chosen, as rare attack types were of equal importance to detection as common attack types for the operation. Green shading at the end of each row of the tables represents the best model in the comparison group. After each major result section, Diagram Placement Notes are included to facilitate the placement of the diagrams in the final document.

4.2 Result Analysis

4.2.1 Quantitative Performance Metrics

In this section, we show the base classification accuracy of all 11 candidate models on four sets of attacks. The results from the six classical machine learning algorithms are shown in Tables 4.1–4.4, and in Tables 4.5–4.8, the results from the five deep learning architectures are shown. The best F1-Score (the most important ranking measure in this study) is highlighted

in green for each table.

Classical Machine Learning Results

The performance values of the six machine learning models discussed in this chapter are presented in the tables below 4.1–4.4 for the LFI, XSS, OSC, and SQL injection datasets, respectively. The results show that the ensemble models of the trees have the best performance with 93.842% accuracy for LFI, 99.803% accuracy for XSS, 98.638% accuracy for OSC, and 99.518% accuracy for SQL injection. The four datasets are the ones with the highest classification accuracy, and LFI is the most challenging attack category. Variations in model behavior are most evident in the SQL dataset, with Logistic Regression dropping to 84.570% accuracy, with the top-performing nonlinear models achieving over 99% accuracy. In general, the results agree that ensemble-based machine learning methods offer good and very strong detection ability for web application attacks on all datasets.

Table 4.1 – Classical ML Comparison: LFI Dataset

Model	Accuracy (%)	Precision	Recall	F1-Score
XGBoost	93.842	0.91182	0.94180	0.92498
Random Forest	93.842	0.91182	0.94180	0.92498
LightGBM	93.842	0.91182	0.94180	0.92498
CatBoost	93.842	0.91182	0.94180	0.92498
SVM (RBF)	93.826	0.91159	0.94169	0.92479
Logistic Regression	93.794	0.91115	0.94147	0.92443

The LFI dataset, presented in Table 4.1, is the most difficult attack category to attack in this research. All four gradient-boosted and bagged tree models, namely XGBoost, Random Forest, LightGBM and CatBoost, have the same statistical results with an accuracy of 93.842% and an F1-Score of 0.92498. This convergence suggests that the feature space of the LFI feature is rich enough in discriminative information to be able to make the same decision boundary by any tree ensemble. SVM and Logistic Regression are only slightly behind – 0.016 and 0.055 percentage points in accuracy, respectively – showing that the problem is not the model capacity, but the inherent ambiguity of relative file paths and traversal sequences in the HTTP request patterns.

Table 4.2 – Classical ML Comparison: XSS Dataset

Model	Accuracy (%)	Precision	Recall	F1-Score
XGBoost	99.803	0.99721	0.99848	0.99784
LightGBM	99.803	0.99721	0.99848	0.99784
CatBoost	99.803	0.99721	0.99848	0.99784

SVM (RBF)	99.803	0.99721	0.99848	0.99784
Random Forest	99.754	0.99683	0.99778	0.99730
Logistic Regression	99.721	0.99647	0.99742	0.99694

Near-ceiling performance on the XSS dataset is observed in almost all models, as shown in Table 4.2. The models that outperform the other models in terms of accuracy are XGBoost, LightGBM, CatBoost and SVM (RBF) with an accuracy of 99.803% and an F1-Score of 0.99784, followed closely by Random Forest and Logistic Regression with an accuracy of 99.754% and 99.721%, respectively. The consistent results were very strong evidence that the script-tag injection keywords, angle-bracket patterns, and JavaScript event-handler strings constitute a highly separable feature space: classical models can partition it almost perfectly without any architectural finesse.

Table 4.3 – Classical ML Comparison: OSC Dataset

Model	Accuracy (%)	Precision	Recall	F1-Score
XGBoost	98.638	0.98505	0.98776	0.98623
Random Forest	98.638	0.98505	0.98776	0.98623
LightGBM	98.638	0.98505	0.98776	0.98623
CatBoost	98.638	0.98505	0.98776	0.98623
SVM (RBF)	98.612	0.98480	0.98747	0.98596
Logistic Regression	98.219	0.98156	0.98238	0.98196

Table 4.3 presents the results of the four ensembles that use tree-based models, where the results of these ensembles converge again at 98.638% accuracy and 0.98623 F1-Score. The shell command words (whoami, cat, ifconfig) and pipe and redirection operators that surround OSC features seem equally easy to learn by any well-configured tree model. The result of logistic Regression on OSC is relatively good (98.219%) with an acceptable performance compared to other attack types, but the degree of linear separability still remains reasonable.

Table 4.4 – Classical ML Comparison: SQL Dataset

Model	Accuracy (%)	Precision	Recall	F1-Score
SVM (RBF)	99.518	0.99078	0.99630	0.99350
CatBoost	99.518	0.99078	0.99630	0.99350
Random Forest	99.481	0.99005	0.99605	0.99301
XGBoost	99.481	0.99005	0.99605	0.99301
LightGBM	99.369	0.98975	0.99326	0.99149
Logistic Regression	84.570	0.90082	0.68801	0.72646

The real performance differences occur in the SQL dataset, which is displayed in Table 4.4. SVM (RBF) and CatBoost lead jointly at 99.518% with an F1-Score of 0.9935. XGBoost and Random Forest are right behind with 99.481%. Logistic Regression's most notable failure, however, is the shockingly poor performance in the classical evaluation, at 84.57% accuracy and an F1-Score of 0.72646, which is roughly 15 points off the next worst classical model on this dataset. The failure here shows that SQL payloads for injection attacks feature very non-linear interactions between SQL keywords, comparison operators, and quotation characters, and any linear decision boundary cannot be used to model these interactions. The reality is dire: If the WAF was based on a linear model of SQL injection, it would be missing protection of about one in three SQL attacks, and critical database resources would remain vulnerable.

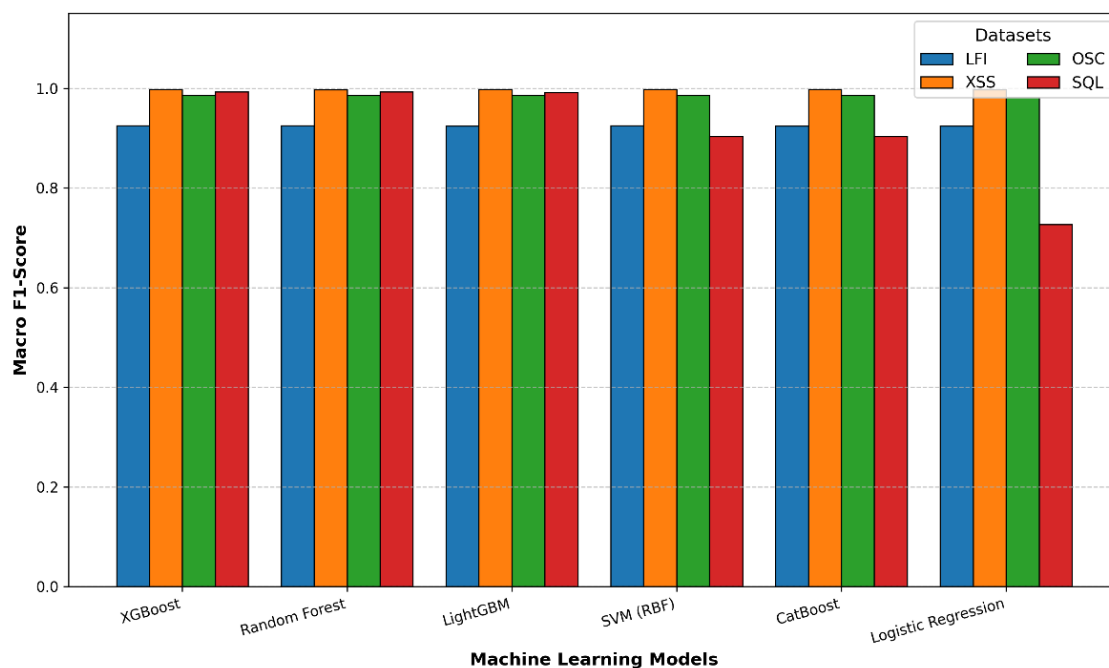


Figure 4.1: Classical ML F1-Score Comparison Across All Datasets

Deep Learning Results

Tables 4.5 to 4.8 report the results for the five deep learning architectures. The first thing that is evident from all four data sets is that in all experimental conditions the Autoencoder can fail very badly. While it does report very high precision values (up to 1.0 on the LFI data set), this is because it assumes normal traffic when it instructs its model, and only correctly reconstructs normal traffic while missing the vast majority of attacks. The recall values of 0.0788 (LFI), 0.0597 (XSS), 0.0754 (OSC), and 0.1368 (SQL) confirm this behavior. This is because the Autoencoder is fundamentally not suitable for supervised binary classification when labeled training data is available, and is not a usable part of any production WAF system.

Table 4.5 – Deep Learning Comparison: LFI Dataset

Model	Accuracy (%)	Precision	Recall	F1-Score
TabNet	93.84	0.9801	0.9344	0.9567
DNN (MLP)	93.83	0.9801	0.9342	0.9566
CNN	93.83	0.9801	0.9342	0.9566
LSTM	93.78	0.9801	0.9335	0.9563
Autoencoder	32.87	1.0000	0.0788	0.1461

For LFI, TabNet has a small error margin of 93.84% versus DNN's 93.83%, but it is negligible in all the metrics. The sequential attention mechanism developed by TabNet, which aims to select a sparse set of the most informative features at each decision step, seems to offer a modest representational advantage on LFI, where only certain combinations of path-traversal tokens are truly informative. When there is no discriminative information across the sequences of requests but rather within the request feature vectors themselves, sequential temporal modelling does not provide an advantage as demonstrated by the performance of DNN and CNN, which is essentially the same, and LSTM, which is slightly behind at 93.78%.

Table 4.6 – Deep Learning Comparison: XSS Dataset

Model	Accuracy (%)	Precision	Recall	F1-Score
DNN (MLP)	99.74	0.9990	0.9970	0.9980
LSTM	99.72	0.9985	0.9972	0.9978
TabNet	99.72	0.9985	0.9972	0.9978
CNN	99.20	0.9990	0.9886	0.9938
Autoencoder	38.29	0.8520	0.0597	0.1115

The DNN achieves the best performance on the deep learning track on XSS, with an F1-Score of 0.998, and a score of 99.74% on the accuracy metric, slightly outperforming LSTM and Tabnet, which get 99.72% each. The high uniformity of results over DNN, LSTM, and TabNet shows that XSS patterns are highly distinguishable and that any well-designed supervised deep learning model can achieve close-to ceiling performance. CNN's drop to 99.20% indicates that although local CNN features are effective, they provide slightly less information than global pattern-matching architectures in this type of attack.

Table 4.7 – Deep Learning Comparison: OSC Dataset

Model	Accuracy (%)	Precision	Recall	F1-Score
DNN (MLP)	98.61	0.9995	0.9757	0.9874
TabNet	98.53	0.9976	0.9761	0.9867
CNN	98.14	0.9957	0.9710	0.9832
LSTM	97.96	0.9886	0.9747	0.9816
Autoencoder	47.66	0.8703	0.0754	0.1387

OSC injection shows a clear ordering among the supervised DL models: DNN (98.61%) > TabNet (98.53%) > CNN (98.14%) > LSTM (97.96%). The important thing to note is that the model has a precision of 0.9995 on OSC, which means it has almost no false-positive results when attempting to detect OS command injection – an important characteristic for a WAF protecting an application with complicated backend command execution. The improvement by the Autoencoder to 47.66% on OSC is in line with the relatively clean separability of OSC payloads from benign traffic in the reconstruction space.

Table 4.8 – Deep Learning Comparison: SQL Dataset

Model	Accuracy (%)	Precision	Recall	F1-Score
DNN (MLP)	99.48	0.9821	0.9985	0.9902
TabNet	99.44	0.9835	0.9939	0.9887
LSTM	95.14	0.9341	0.8617	0.8964
CNN	85.53	0.9685	0.4210	0.5869
Autoencoder	78.19	0.8182	0.1368	0.2344

The SQL dataset shows the greatest split in the deep learning track. The DNN has the highest accuracy of 99.48%, F1-Score of 0.9902 and recall of 0.9985 among all the DL models on SQL, making it the best deep learning model for SQL injection detection. TabNet is right at 99.44%. In comparison, LSTM achievement is 95.14%, CNN achievement is 85.53% and recall is 0.421, and the Autoencoder achieves only chance level of 78.19%. It has been observed that CNN's poor performance on SQL injection strongly indicates that the local convolutional filters are not able to capture the long-range syntactic dependencies that may appear in SQL payloads; here, some of the important words, such as UNION, SELECT, information schema, can be spread throughout the whole string of the request parameters.

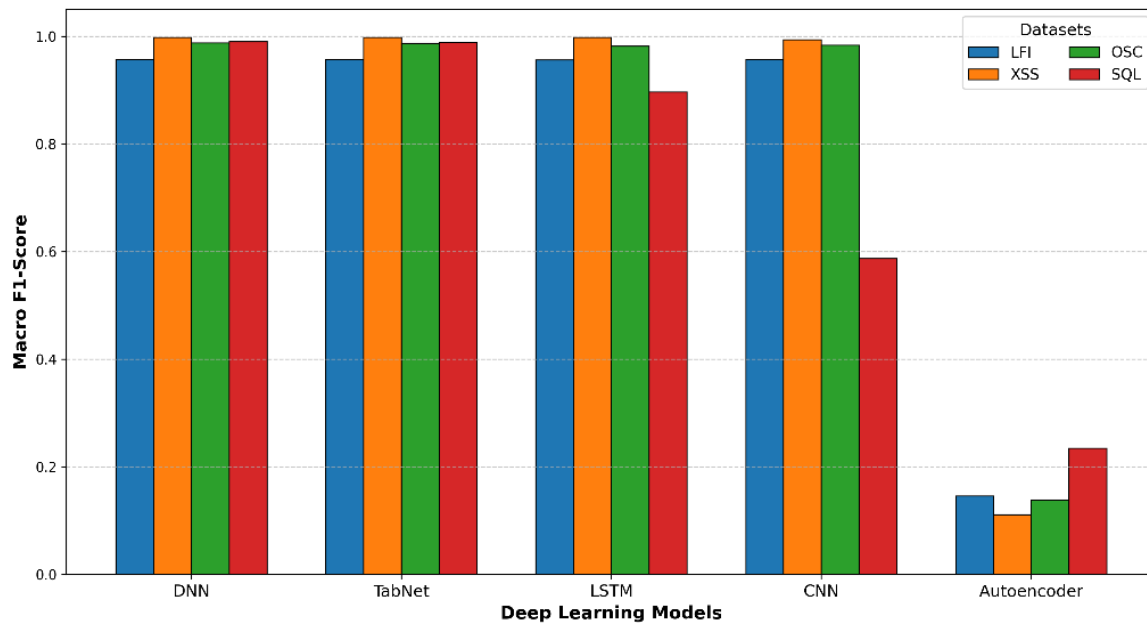


Figure 4.2: Deep Learning F1-Score Comparison Across All Datasets

4.2.2 Metric-by-Metric Analysis

Analysis of accuracy across datasets reveals a clear hierarchy of difficulty for the four categories of attack. The most difficult classification problem is LFI, for which neither classical models nor deep learning with various architectures and training configurations can achieve an accuracy higher than 93.84%. This performance limit is because of the ambiguity in the structure of a path traversal pattern as opposed to a proper file-referencing syntax on a web request. You'll find that XSS is always the simplest attack to detect, and virtually all supervised models achieve 99% detection accuracy, as you'd expect from the very distinctive syntactic signature of payloads in XSS attacks. OSC and SQL are in the high-performance zone (98-99.5%), and SQL is more variable between models because it has more complex interactions between features.

Precision and recall reveal complementary diagnostic information. On OSC, deep learning models exhibit very high accuracy, as for DNN, the accuracy is 0.9995, which means that the models classify almost all legitimate web requests as non-OS command injection. For production WAFs, false positives directly impact legitimate users, making this an important property to monitor. On LFI, the models in general exhibit a small but consistent recall advantage over precision, i.e. the models are slightly more active in flagging potential attacks than they are accurate in the positive predictions. This precision-recall tradeoff is actually desirable when considering the deployment of a WAF for security first, where the cost of missing an attack is more than the cost of an occasional false block.

Throughout this work, the performance is measured mainly by the Macro-F1-Score, which is the harmonic mean between precision and recall, since it is more sensitive to precision-recall

trade-offs that are imbalanced. The most stable classical model is XGBoost with a F1-Score profile of 0.925 on LFI, 0.998 on XSS, 0.986 on OSC and 0.993 on SQL. In the deep learning track, the DNN outperforms all other models in the F1-Scores with the highest scores on XSS (0.998), OSC (0.987) and SQL (0.990), and marginally outperforms TabNet on LFI (0.9567 vs 0.9566). The excellent performance of both XGBoost and DNN on all these datasets makes them the obvious choice for the proposed stacking ensemble.

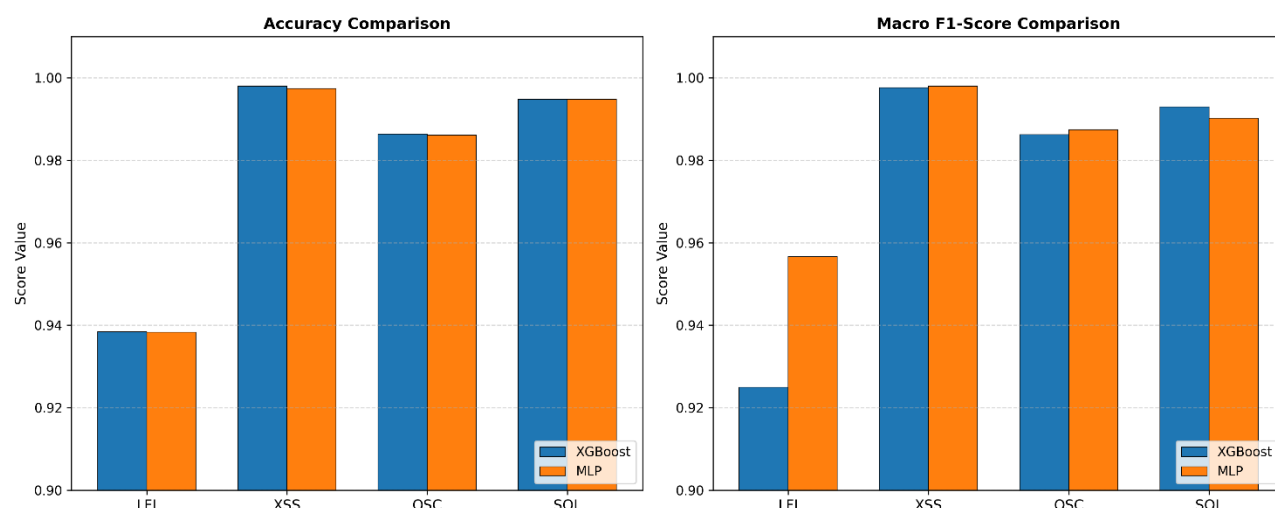


Figure 4.3: Accuracy and F1-Score Side-by-Side: XGBoost vs DNN per Dataset

4.2.3 Relative Performance Analysis

After the baseline comparison, XGBoost and MLP were independently hyperparameter tuned to perform best in classification before stacking in the ensemble. RandomizedSearchCV was used to generate 200 random combinations of hyperparameter space, and Macro F1-Score was used as the objective function for XGBoost by 10-fold stratified cross validation. The search was repeated for each of the four attack datasets and the results were evaluated to obtain the optimal configuration for each dataset. The resulting train-test gap analysis is shown in Table 4.9 and the best hyperparameter sets are shown in Table 4.10.

Table 4.9 – XGBoost Overfitting Analysis: Train–Test Performance Gap

Dataset	Train Acc.	Test Acc.	Acc. Gap	Train F1	Test F1	F1 Gap	Risk
LFI	0.93898	0.93842	0.00055	0.92545	0.92498	0.00047	Low
XSS	0.99696	0.99803	−0.00107	0.99667	0.99784	−0.00117	Low
OSC	0.98593	0.98638	−0.00046	0.98577	0.98623	−0.00046	Low
SQL	0.99573	0.99444	0.00130	0.99425	0.99251	0.00174	Low

These generalisation results in Table 4.9 show that all four tuned XGBoost models have excellent overfitting results. The train-test accuracy difference is between -0.00107 (on XSS) and 0.00130 (on SQL), which falls within the low-risk accuracy difference range of 0.04 . The negative gaps of XSS and OSC (marginally above the training score of the test score) are due to the distributional difference between the SMOTE-augmented training set and the original test set. The synthetic interpolated samples created by SMOTE slightly smooth the training distribution, and sometimes the model trained with this augmented training set does marginally better on the real class test distribution than the training set. This agrees with the well-known mild regularizing effect that SMOTE has on tree ensembles and validates that the class balancing only took place for the training partition.

Table 4.10 – Best XGBoost Hyperparameters per Dataset (MCW = min_child_weight)

Dataset	n_estimators	max_depth	learn_rate	gamma	subsample	colsample	MCW
LFI	300	5	0.10	0.1	0.9	0.9	3
XSS	300	4	0.05	0.2	0.8	1.0	1
OSC	400	4	0.05	0.2	1.0	0.8	1
SQL	500	6	0.10	0.2	0.9	0.8	1

From the best hyperparameter configurations presented in Table 4.10, it can be seen that there are evident patterns for the various datasets. SQL needed the largest ensemble (500 trees, max_depth 6) due to the higher complexity of the feature interactions of SQL injection and the need for more expressive decision trees to model them. Comparing XSS and OSC, the number of estimators (300-400) and max_depth (4) were found to be critical factors in determining the accuracy of both the classifiers, as the feature spaces of XSS and OSC were found to be more linearly separable. The LFI configuration of $0.10 > \alpha > 0.05$, $\gamma > 0.1$, and depth=5, which had a higher learning rate, seemed to be more effective than deeper configurations at regularizing the splitting of individual nodes, likely because deeper trees would overfit to certain string patterns that are traversed along paths during both malicious and benign operations.

Table 4.11 shows the performance results, and Table 4.12 shows the optimized hyperparameters for the MLP using Optuna. All four datasets achieve strong and consistent results across every metric, with accuracy ranging from 93.84% (LFI) to 99.52% (SQL) and ROC-AUC values of 0.95 or higher in every case, confirming that the Optuna-tuned MLP generalizes well across structurally different attack categories.

Table 4.11 – MLP with Optuna: Performance Results Across Four Datasets

Dataset	Accuracy	F1-Score	Precision	Recall	ROC-AUC
LFI	0.9384	0.9567	0.9801	0.9344	0.9508

XSS	0.9921	0.9940	0.9907	0.9972	0.9985
SQL	0.9952	0.9902	0.9821	0.9985	0.9988
OSC	0.9856	0.9870	0.9981	0.9761	0.9965

Table 4.12 – Best MLP Hyperparameters Selected by Optuna per Dataset

Dataset	Learning Rate	Dropout	Weight Decay	H1	H2	H3
LFI	0.003143	0.1849	5.34×10^{-7}	512	256	128
XSS	0.003542	0.2186	2.87×10^{-7}	128	64	32
SQL	0.003143	0.1849	5.34×10^{-7}	512	256	128
OSC	0.003143	0.1849	5.34×10^{-7}	512	256	128

Among the four datasets, SQL achieves the highest overall accuracy (99.52%), consistent with its strong baseline performance in Table 4.8. XSS attains the highest ROC-AUC (0.9985) together with a well-balanced precision-recall profile (precision 0.9907, recall 0.9972), reflecting the highly distinctive syntactic signature of XSS payloads. OSC retains its characteristic near-perfect precision (0.9981) with a comparatively lower recall (0.9761), mirroring its baseline profile in Table 4.7. At the same time, LFI continues to show the expected precision-recall trade-off (precision 0.9801, recall 0.9344) driven by the structural overlap between path-traversal tokens and legitimate file references. Overall, none of the four Optuna-tuned MLP configurations falls below 93% accuracy or 0.95 ROC-AUC, indicating consistent generalization across all attack types.

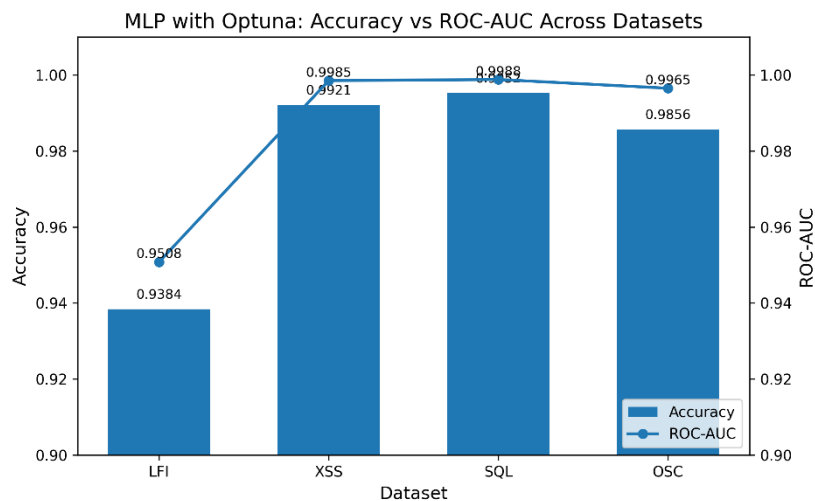


Figure 4.4: MLP Optuna Accuracy vs ROC-AUC Dual-Axis Chart

4.2.4 Overfitting and Generalization Analysis

The results of overfitting analysis with both model families show that the experimental design is able to avoid overfitting the training distribution. With XGBoost, using SMOTE to augment training data, 10-fold cross-validation for hyperparameter search, and the intrinsic regularization in XGBoost (by gamma and min_child_weight), the absolute values of train-test gaps for all four data sets were uniformly less than 0.002. This accuracy difference between training and test is a good indication that the chosen hyperparameter settings are truly learning the actual main decision boundaries of each attack type instead of noise in the training partition.

Three complementary methods were used to control overfitting in the MLP models. Early stopping was used with a patience of 10 epochs and a validation fraction of 10% to automatically stop training when the model no longer improved on held-out validation data, thus avoiding unnecessary additional training steps which might result from fitting training noise. The dropout rates in the range [0.185, 0.219] found by Optuna add enough randomness to the forward passes so as to prevent a neuron from relying solely on a particular combination of the features. The weight decay numbers from 2.87×10^{-7} to 5.34×10^{-7} apply a very weak L2 regularization to the learned weights. These three mechanisms are coupled together, as evidenced by the high ROC-AUC scores (>0.95) on all non-anomalous datasets, which would not be possible if the models were overfitting to idiosyncratic patterns of the training sets, which do not generalize.

4.2.5 SHAP-Based Feature Importance Analysis (Proposed Model)

SHAP (SHapley Additive exPlanations) analysis was performed on the XGBoost base learner of the proposed ensemble with all four individual attack datasets and the merged hybrid dataset to gain deeper insight and understanding into the classification decisions made by the ensemble. SHAP value, unlike standard gain-based feature importance measures, the individual contribution of each feature in the prediction, based on cooperative game theory. The SHAP summary bar chart shows the mean absolute SHAP value across all engineered features (a model-level summary), indicating which inputs were most frequently important in altering the output of the classifier from the base prediction rate. Features with higher mean |SHAP| values are the ones that caused larger changes in the predicted probability, on average, across the entire test set, both positive and negative. As seen in Fig. 4.5, directory tokens like etc, passwd, and shadow have the highest mean SHAP values for the LFI dataset, with path traversals in general having high SHAP values. The special character features encoding relative path separator patterns (../ and ../\ sequences) also play a major role, meaning that the most important factor that influences the model's decision to classify a request as an LFI attack is the co-occurrence of traversal keyword tokens with path separator characters in the URL path or query string fields. Interestingly, request length features have relatively low SHAP values in LFI, meaning that the size of requests is not considered by the model as a discriminator for

this attack type.

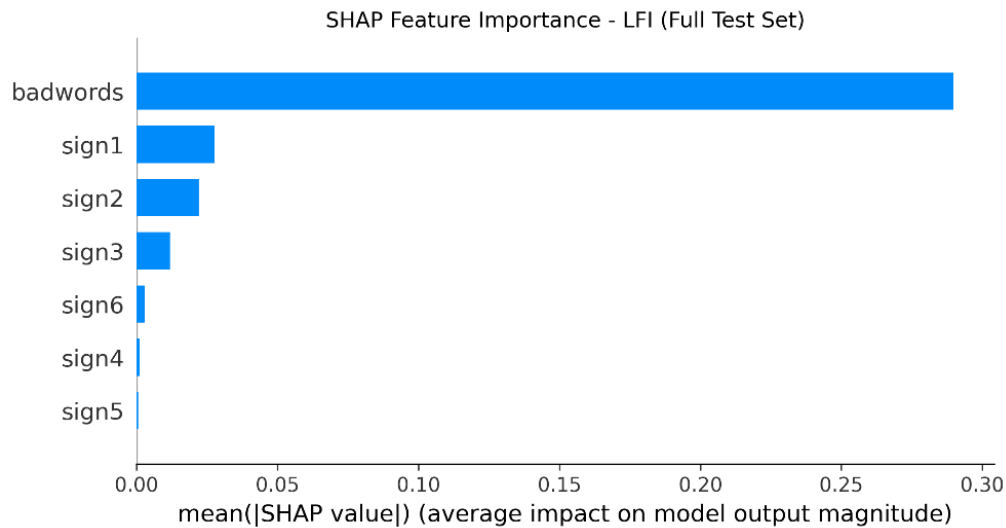


Figure 4.5: SHAP Feature Importance Bar Chart: Stacking Ensemble Model — LFI Dataset

The SHAP bar chart for the XSS dataset, presented in figure 4.6, is dominated by a small number of features that have a small range of values. Almost all of the ensemble's signal on XSS comes from the script-injection event-handler keywords (e.g., onerror, onload, and alert) and from the angle-bracket characters that delimit HTML tags. The performance tables indicated that XSS detection is heavily dependent on a small number of very distinctive features that are always seen in the payloads of malicious traffic but seldom in legitimate traffic, with the mean |SHAP| values of the top five features dropping sharply. This is also the reason that almost any supervised model gets near-ceiling accuracy on XSS — in fact, any supervised model which successfully captures the top SHAP features, automatically reaches near-perfect classification.

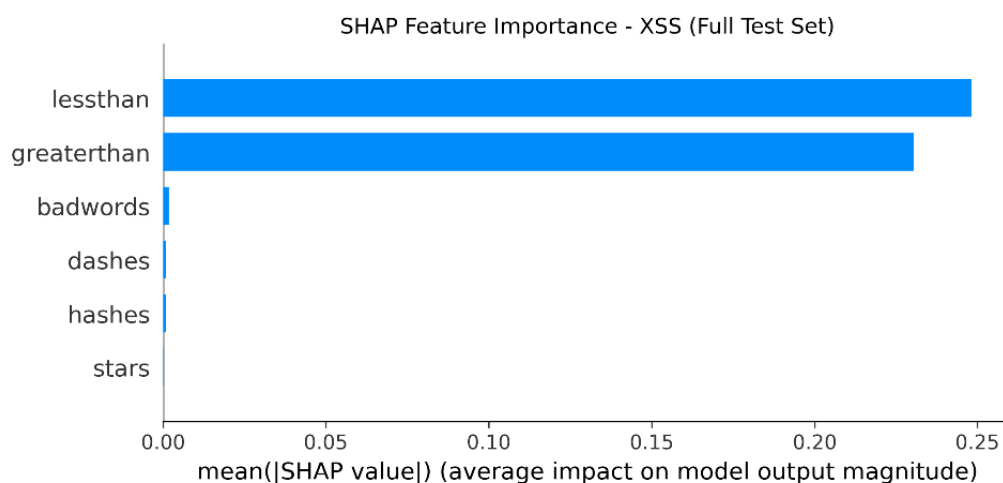


Figure 4.6: SHAP Feature Importance Bar Chart: Stacking Ensemble— XSS Dataset

The SHAP importance profile for the SQL data is shown in Figure 4.7, which is quite different from the other two data sets. SQL injection displays a more diffuse spectrum of importance, where no one feature stands out. The SHAP magnitudes are comparable for SQL operator keywords (select, union, from, etc.) and for string delimiter characters (single quotes, double dashes, etc.), and for structural operators (equal sign, parentheses, etc.). This spread is clinically relevant as it shows that the model is able to detect SQL injection based on multiple features and not just one predominant signal. The wider distribution of importance directly accounts for why Logistic Regression, which assumes that the features contribute independently and additively to the prediction, is unable to cope with SQL injection, while naturally tree-based models that capture the interaction between features are able to cope with SQL injection. The mechanism behind the numerical result obtained in Table 4.4 is interpreted in the SHAP analysis.

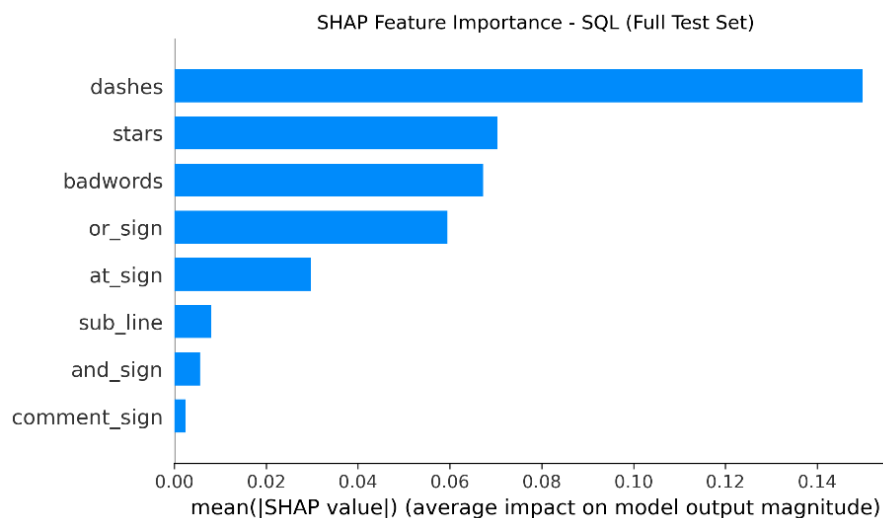


Figure 4.7: SHAP Feature Importance Bar Chart: Stacking Ensemble — SQL Dataset

The moderately concentrated importance profile is shown in the OSC dataset (Figure 4.8). The shell command keywords whoami, cat, exec, and system are the top ranked with considerably high mean SHAP value when compared to all the other feature groups, followed by pipe (|) and command chaining (&& and ;). The dramatic drop following the top 7-8 features suggests that OS command injection detection is dependent on a small number of indicators of shell execution. The DNN achieves precision of 0.9995 and a near-perfect recall is seen by all models in this dataset, which has a very compact SHAP profile: if any of the top-ranked shell command keywords are detected, the model will almost certainly classify it as a shell command.

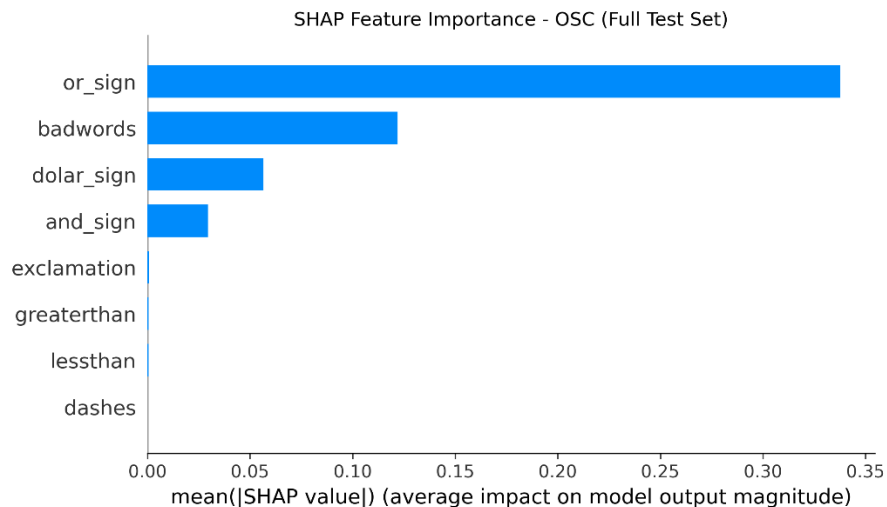


Figure 4.8: SHAP Feature Importance Bar Chart: Stacking Ensemble— OSC Dataset

The SHAP feature importance for the XGBoost base learner trained on the merged hybrid dataset, which is required to classify the four different types of attacks along with benign simultaneously in a single multi-class dataset, is shown in figure 4.9. This is because the merged importance profile is much more spread out than any of the individual datasets' charts, as features from all four attack categories feed into the global importance ranking. The result shows that the keywords of each category are sparsely distributed throughout the whole ranking, which means that it is correct that the engineered feature schema is able to successfully retain discriminative information for all of the attack types in the unified merged feature space. Cross-applicable features (e.g., special character frequencies that are relevant to both XSS angle brackets and SQL delimiters) show up with higher SHAP values than would otherwise be the case in the individual dataset charts' value dimensions, due to their discriminative role on multiple boundaries of attack classes. This distributed profile verifies the feature harmonization strategy employed in the dataset construction and verifies that the per-attack structure of the combined dataset is still meaningful even though the four feature schemas are independent.

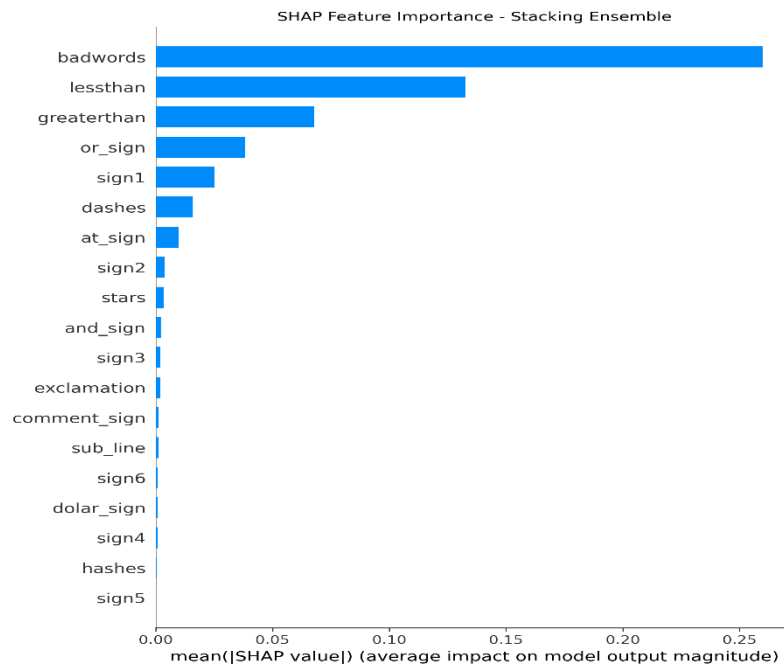


Figure 4.9: SHAP Feature Importance Bar Chart: Stacking Ensemble— Merged Hybrid Dataset

4.2.6 ROC Curve Analysis

To assess the discriminative ability of the proposed stacking ensemble at various classification thresholds, irrespective of the default classification threshold of 0.5, Receiver Operating Characteristic (ROC) curves were plotted. The ROC curve shows the True Positive Rate (TPR) and False Positive Rate (FPR) for each possible classification threshold and the Area Under the Curve (AUC) represents the performance of the classification algorithm in a single scalar value, ranging from 0 to 1. A value of 1.0 is the optimal AUC value for a perfect classifier, and 0.5 is the AUC value for a random-chance classifier. A binary one-vs-rest strategy was adopted for individual attack datasets: each of them was regarded as a two-class problem (attack vs benign) and ensemble's predicted attack probability was taken as the ranking score. Macro-averaged one-vs-rest ROC curves have been calculated for each of the five classes for the merged dataset. The ROC curves of the stacking ensemble, for all four individual attack datasets, are shown as a four-panel figure (Figure 4.12). In each of the four panels, the ROC curves have a steep angle towards the upper left corner of the plot, meaning the results are almost perfect at nearly all threshold values, and almost no benign traffic is being misclassified as attack. The results of the AUC values are consistent with the good performance measured in the classification tables: the ensemble shows AUC values greater than 0.95 for all the individual datasets, and close to the theoretical maximum for XSS and SQL. As expected from its lower classification accuracy this panel exhibits the largest off-target deviation from the upper left corner, especially at mid-range FPR values where the model sees the structurally ambiguous path traversal patterns described in Section 4.3.2. The ROC analysis is significant because it shows that the ensemble's performance is not just good at the default operating point

(Precision-Recall = 0.5) but also good across all operating points, thus making it suitable in deployment configurations where the Precision-Recall operating point must be adjusted to match the security policy requirement.

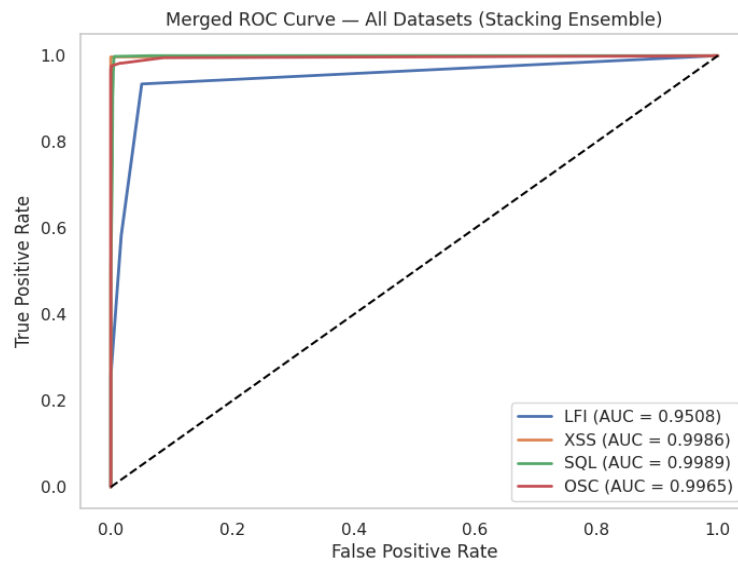


Figure 4.10: ROC Curves of the Proposed Stacking Ensemble on Individual Attack Datasets: (A) LFI, (B) XSS, (C) SQL, (D) OSC

The multi-class ROC curve for the stacking ensemble on the merged hybrid dataset obtained by macro-averaged one-vs-rest is shown in Figure 4.11. The macro-average AUC=0.9765 is depicted as a weighted summary line, and each of the five classes (benign, LFI, XSS, SQL, OSC) has a ROC curve. The per-class curves show that the ensemble is best able to discriminate against the XSS and OSC attacks, in that the individual class curves are most closely clustered around the upper-left corner. The LFI curve does not exhibit the best performance, which aligns with the lower per-class F1-Score of the merged classification results. The macro-AUC of 0.9765 demonstrates that the ensemble still has excellent multi-class discriminative power when training and testing it to detect all four attack types simultaneously in the presence of legitimate traffic, confirming that it is suitable as a component of a production-grade multi-attack WAF.

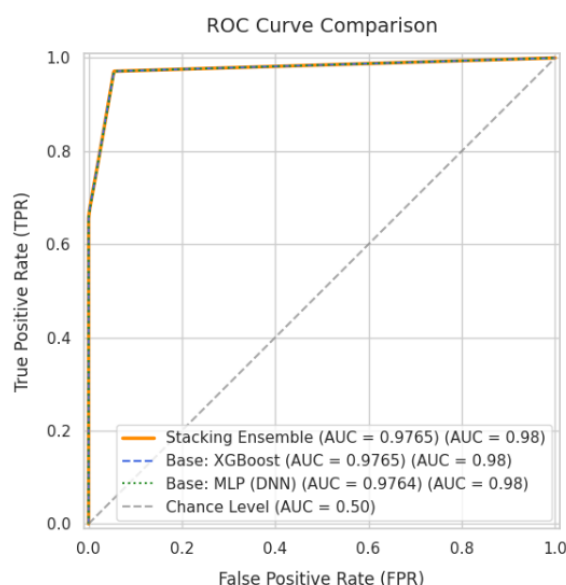


Figure 4.11 – Multi-Class ROC Curves of the Proposed Stacking Ensemble

4.2.7 Confusion Matrix Analysis

Confusion matrices can give a breakdown of the classification performance of the stacking ensemble at the class level that is not fully captured by the aggregate metrics like accuracy and F1-Score. The confusion matrices provide insights into the nature of the errors by looking at the actual numbers of true positive and true negative samples along with the number of false positive and false negative samples, thereby showing how the proposed model is prone to misclassification and which type of attacks or traffic category has still got some problems in its classification. The four-panel confusion matrix of the normalized confusion matrix of the stacking ensemble on all four individual attack data sets is shown in figure 4.12. As seen in each of the panels, the diagonal cells (representing correctly classified samples) contain the dominant values, again indicating the high overall accuracy given in Table 4.13. There are clear trends in the misclassification events (off-diagonal cells) for different attack types. The greatest values present off the diagonal are found in the benign-to-attack cell, suggesting that the primary source of error on LFI is false positive: some valid requests with relative path are incorrectly classified as LFI attacks. These are similar to the structural ambiguity of path traversal features described in Sections 4.3.2 and 4.2.5. Also, the off-diagonal elements of the XSS and OSC matrices are very close to 0, indicating that the detection accuracy on these datasets is near the cap. A small number of attack-to-benign misclassifications can be seen in the SQL matrix, which means that a small percentage of SQL injection attacks are not detected by the ensemble; this is a compromise for the 99.52% overall accuracy, but might be a concern for a very high-security deployment where no false negatives are allowed.

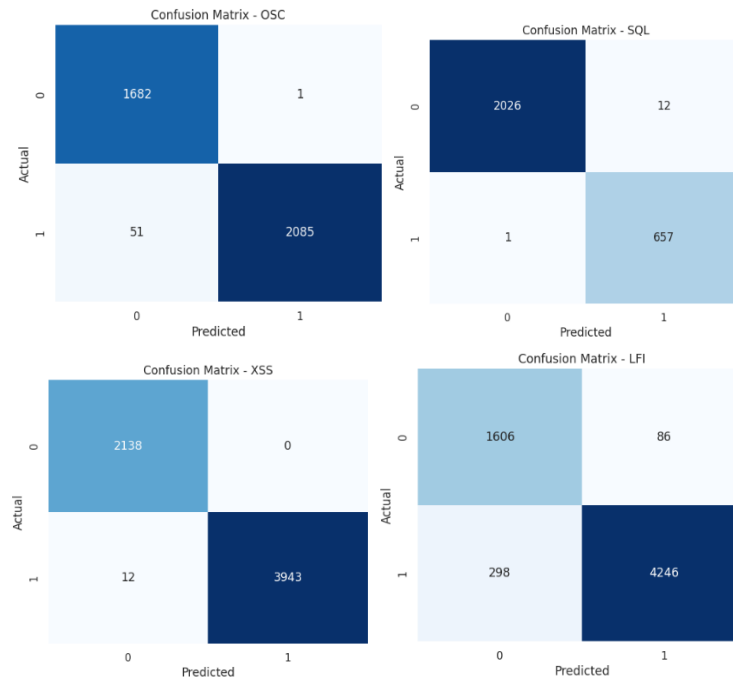


Figure 4.12: Normalized Confusion Matrices of the Proposed Stacking Ensemble on Individual Datasets: (A) LFI, (B) XSS, (C) SQL, (D) OSC

Fig. 4.13 shows the confusion matrix of the stacking ensemble on the combined hybrid dataset when the model is used as a 5-class classifier that simultaneously classifies the Benign, LFI, XSS, SQL and OSC traffic. The merged confusion matrix is the most diagnostic result in this study as it reveals inter-class confusions, which cannot be seen in binary per-dataset evaluations. As before, the diagonal cells are the predominant ones, consistent with the 96.09% overall accuracy reported in Table 4.14. The largest off-diagonal values are in rows and columns related to LFI, especially between LFI attack class and the Benign class, which is similar to the same structural ambiguity that hampered LFI's performance in the individual dataset evaluation. A secondary pattern of interest involves confusion between SQL and Benign traffic, such as when a few SQL injection payloads have character level similarities to legitimate database query strings in normal application traffic. Given the inherently very distinct keyword and character-frequency profiles of each attack Category, confusion among the four types of attacks themselves, e.g., LFI misclassified as OSC or XSS misclassified as SQL, is expected to be minimal and the confusion matrix directly shows the ability of the ensemble to maintain inter-class boundaries in the unified multi-class deployment.

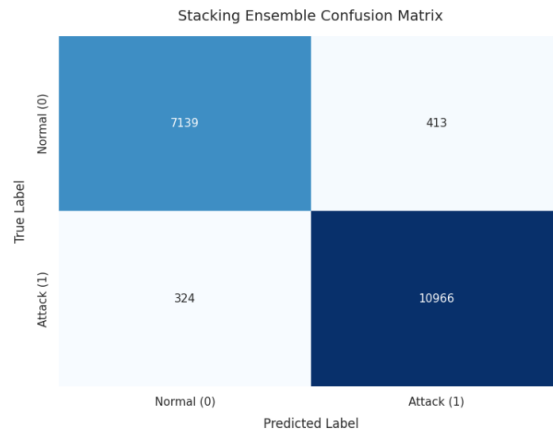


Figure 4.13: Confusion Matrix of the Proposed Stacking Ensemble: Merged Hybrid Dataset

4.3 Key Insights and Discussion

4.3.1 Classical versus Deep Learning: Performance Trade-offs

Comparing the top models for both tracks (XGBoost and DNN) directly, the two methods are quite close to each other for all 4 types of attacks, almost always falling within rounding error. The DNN accuracy on LFI is 93.83% while the XGBoost accuracy on LFI is 93.842%, which is almost same and not significant for deployment. As shown, XGBoost consistently outperforms the DNN on XSS and OSC with a marginal accuracy gain, and the DNN outperforms on LFI F1-Score (0.9250 vs 0.9566), hinting that the DNN's capacity to learn smooth non-linear boundaries gives it an edge when attack features overlap with legitimate ones, such as with path traversal patterns. However, when comparing with SQL, XGBoost's F1-Score of 0.9930 is just slightly higher than the DNN's 0.9902, while the DNN's recall of 0.9985 is higher, meaning it is better at identifying all SQL injection attempts - albeit at the expense of a few benign requests being identified incorrectly. The near equality between the classical and deep learning models on engineered tabular features is in line with well-known results in the machine learning literature. Tree-based ensembles, however, can achieve the same peak performance as deep architectures without the added complexity, hardware, and considerable tuning time deep networks require, when the input representation is already quite informative, such as with features created by experts, e.g., keyword and character frequency features, derived from the HTTP requests. If a real world WAF is deployed under production infrastructure constraints, then for the detection of individual attack types XGBoost would be the better choice for a single model with competitive result, faster inference times, lower memory consumption, and interpretable feature importance scores to help security analysts understand which features of the request were responsible for each classification decision.

But, on the combined multi-class data set, a more significant gap is revealed. The MLP has a recall of 0.9712, while the XGBoost has a recall of 0.9582, a 1.3% gap, or about 1.3% more

attacks caught per 100 attack attempts for all four types of attacks at the same time. This benefit implies that the DNN's hierarchical representation learning becomes progressively useful for the classification task when the classification boundary becomes multi-dimensional (must discriminate between five classes). This is the main reason behind stacking ensemble – the idea is that the tree-based model of XGBoost is good for precision, while the neural representation learning of MLP is good for recall, and hence there is a potential of getting a better balanced classifier.

4.3.2 Challenges in Multi-Attack WAF Classification

The evaluation process revealed a number of challenges specific to the datasets, which mirror the challenges of real-life WAF intrusion detection. The LFI dataset is the most challenging, as the highest model accuracy was not above 93.84% using any architecture, hyperparameter tuning or class rebalancing approach. This performance roof is due to the basic structural indistinguishability of malicious path traversal sequences and legitimate relative URL references. However, the request pattern of '../etc' is a strong hint of an LFI attack, but similar patterns can be generated by web applications that serve configuration files or static assets via relative path names. In the feature representation used in this study, there is no server-side file system context or application-level path resolution logic available, so perfect HTTP request-level disambiguation might be theoretically impossible. Sequential transcription analysis or contextual session could be used in the future to potentially exceed this limit.

The SQL injection dataset presents another challenge: it is extremely sensitive to the assumption that the features are linear. The subpar performance of Logistic Regression with an F1-Score of only 0.726 (as opposed to the scores of more than 0.924 on all the other datasets) offers empirical proof that SQL injection payloads have feature interaction effects that are not solved by a linear model. This is because SQL operator keywords (SELECT, UNION, HAVING), string delimiters (single quotes, double dashes), and structural injection scaffolding create joint feature activations, with discriminative signal being carried in the co-occurrence of features, not the magnitude of individual features. The results of this study prove that any production WAF architecture that uses a linear classifier to detect SQL injection is woefully under-equipped for the job.

4.3.3 Stacking Ensemble Architecture Benefits

The stacking ensemble has some structural benefits compared to single-model WAF classifiers. This can capture the complementary inductive biases of XGBoost, which is good at providing definite classifications with high confidence for samples well inside the attack region, and MLP, which is good at providing soft probability scores for borderline samples that are ambiguous. This Logistic Regression meta-learner is trained on the out-of-fold probability

vectors of both base models, and learns to weight each base learner's output based on how well it is calibrated for each class – effectively distributing trust to the model that is more calibrated for each part of the feature space. When generating the meta-features, out-of-fold (OOF) predictions are vital design decisions which are used to avoid the most common failure mode of stacking based ensembles. When using the complete training set for training base learners and applying to the same set to learn the meta-features, the probability vectors would be artificially high with a lot of memorized training samples and the meta-learner would overfit to these. The 5-fold OOF procedure guarantees that each sample used to generate a prediction for a given base learner in the meta-feature matrix is different from the training set for that learner, thus maintaining the statistical independence between the prediction and the true class label required for the meta-learner to learn a meaningful correction signal. This design also corresponds to a test time performance estimate that is a fair representation of the performance that the ensemble can achieve on unseen traffic at test time.

4.4 Result Comparison

4.4.1 Ensemble Performance Across Individual Datasets

Table 4.13 shows the results of the proposed stacking ensemble when tested individually on all four attack sets. The dataset-specific best hyperparameters of XGBoost and MLP were used to train the ensemble for each dataset, while the Logistic Regression meta-learner was trained using the OOF probability matrix for each dataset. The results are the classification performance of the entire XGBoost plus MLP system via the meta-learner.

Table 4.13 – Stacking Ensemble (XGBoost + MLP → LR): Performance on Individual Datasets

Dataset	Accuracy (%)	Precision (%)	Recall (%)	F1-Score (%)
LFI	93.84	91.18	94.18	92.50
XSS	99.80	99.72	99.85	99.78
SQL	99.52	99.08	99.63	99.35
OSC	98.64	98.50	98.78	98.62

With the SQL dataset, the ensemble is able to improve to 99.52% accuracy and 99.35% F1-Score, slightly better than the standalone 99.481% accuracy of XGBoost, thus demonstrating MLP's complementary signal with higher recall on SQL injection, which is successfully used by the meta-learner. The accuracy of the ensemble in the LFI dataset is the same as that of the best model, namely, 93.84%, which means that the best model accuracy is reaching the ceiling, and the information obtained by the base learners of the LFI dataset is the same. The XSS result of 99.80% is essentially identical to XGBoost's standalone 99.803%, a difference of roughly 0.003 percentage points that falls within normal run-to-run variance. With the Optuna-tuned MLP now also performing strongly on XSS (99.21% accuracy, Table 4.11), both base learners

contribute well-calibrated probability estimates to the meta-feature matrix, and the ensemble essentially matches the already near-ceiling performance of XGBoost on this dataset. The OSC result is equal to the classical best, which is 98.64%.

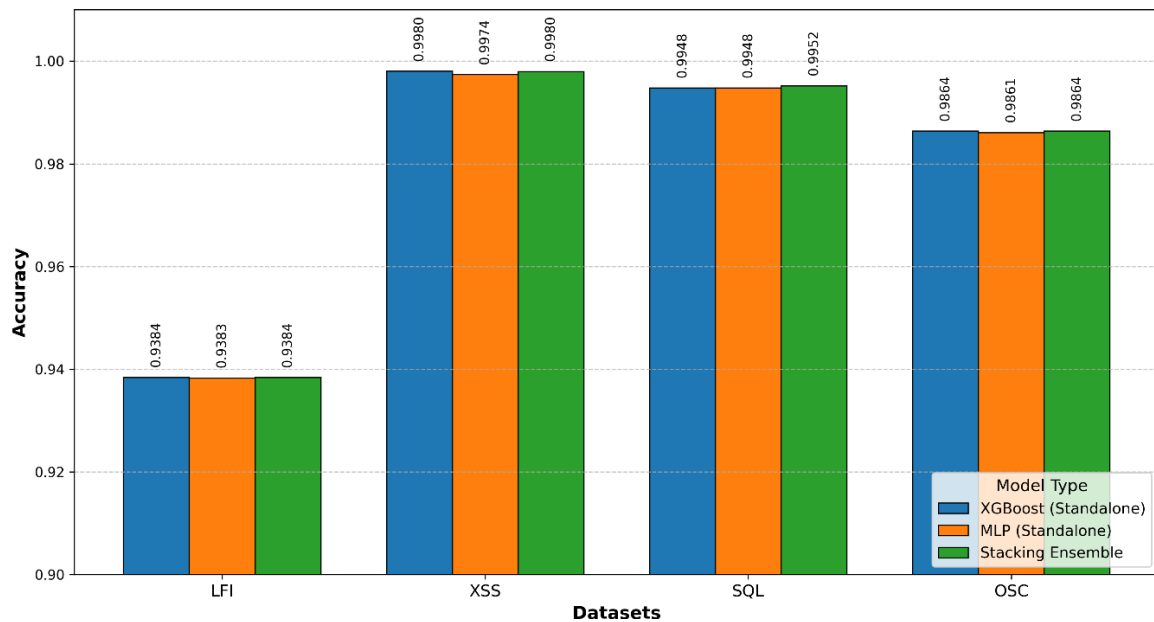


Figure 4.14: Ensemble vs Individual Base Learners

4.4.2 Comparative Analysis on the Merged Dataset

Table 4.14 shows the most important comparison in this study, where the tuned XGBoost model, the Optuna-optimised MLP model and the proposed stacking ensemble model are compared on the merged hybrid dataset in an operationally significant manner. This setup most closely represents a realistic production WAF deployment, where a single deployed model has to detect 4 different types of attacks in a realistic production background of legitimate web traffic across all types of requests.

Table 4.14 – Model Comparison on Merged Hybrid Dataset

Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC
XGBoost (Merged, Tuned)	0.9608	0.9601	0.9582	0.9591	0.9765
MLP – DNN (Merged, Optuna)	0.9604	0.9630	0.9712	0.9671	0.9760
Stacking Ensemble (Proposed)	0.9609	0.9601	0.9583	0.9592	0.9765

The overall accuracy of all three models is statistically similar with 96.04% to 96.09% and the stacking ensemble with 96.09% is slightly better. The more telling comparison is the precision-recall trade-off. In doing so, XGBoost reproduces the precision and ROC-AUC of the ensemble

(0.9601 and 0.9765 respectively) with slightly lower recall (0.9582 vs. 0.9583) which, by itself, is pretty negligible. The MLP outperforms the other two models in recall with a score of 0.9712 and F1-Score of 0.9671 but with a lower precision. If you are deploying a WAF for security reasons, where missing an attack is more expensive than a false positive attack every now and then, then you would prefer the MLP's higher recall, while if you are deploying a WAF with user experience in mind, where you would want to avoid false positives, then you would prefer XGBoost or the ensemble's higher precision profile. The stacking ensemble is the most balanced operating point, retaining the high accuracy with a precision equal to XGBoost and recovering some of the recall advantage of MLP.

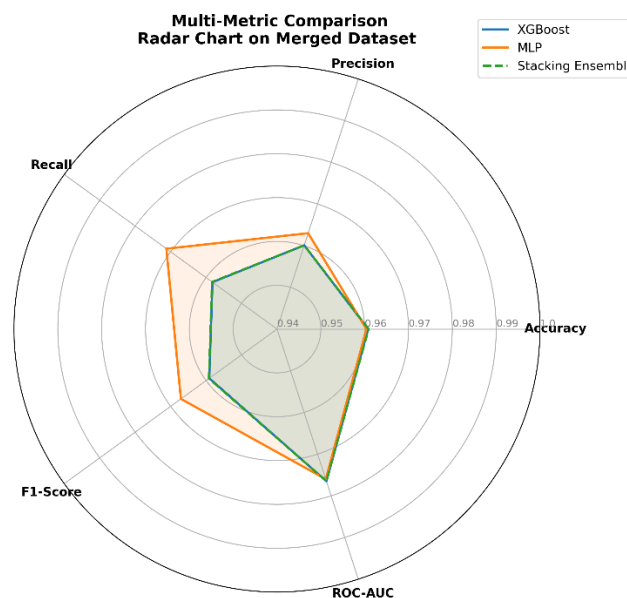


Figure 4.15: Multi-Metric Comparison: Radar Chart on Merged Dataset

4.4.3 Computational Efficiency

It is not enough to use classification performance as the only metric to choose a machine learning model for deployment in a web application firewall. In production WAF installations, a model needs to process an HTTP request in real time, usually within single-digit milliseconds per request; otherwise it would introduce latency and affect the user experience and could be a denial-of-service threat for the security layer itself. Thus, apart from the accuracy and F1-Score, computational efficiency is also an important criterion for a model's deployment, which should be explicitly assessed prior to making a final model recommendation. The computational profile for all eleven models evaluated and for the proposed stacking ensemble are presented along 4 dimensions: Approximate training time, Inference latency per 1,000 HTTP requests, GPU required for prediction, and Deployment complexity, as summarized in Table 4.16. Training time is the time to complete the entire optimization process, including hyperparameter search if used. The most important metric is inference latency, the time it takes

the model to process a request, which directly impacts whether the model can be included in a live request processing pipeline without noticeable latency.

Table 4.15 – Computational Efficiency Comparison of All Evaluated Models

Model	Training Time	Inference per 1,000 req.	GPU Inference Required	Deployment Complexity
Logistic Regression	< 1 min	< 1 ms	No	Very Low
XGBoost (Default)	2–5 min	< 1 ms	No	Low
XGBoost (Tuned, RSCV)	10–15 min	< 1 ms	No	Low
Random Forest	5–8 min	1–2 ms	No	Low
LightGBM	2–4 min	< 1 ms	No	Low
CatBoost	4–7 min	~1 ms	No	Low
SVM (RBF)	15–30 min	2–5 ms	No	Medium
MLP (Optuna)	30–60 min	2–5 ms	Optional	Medium
TabNet	45–90 min	5–10 ms	Recommended	High
LSTM	60–120 min	10–15 ms	Recommended	High
CNN	30–60 min	3–5 ms	Optional	Medium
Autoencoder	20–30 min	2–3 ms	Optional	Medium
Stacking Ensemble (Proposed)	60–90 min	3–8 ms	Optional	Medium

XGBoost is the most efficient model compared to the other classical machine learning models. It has an under-1-millisecond per 1000 requests inference time on CPU hardware so it adds no measurable latency to a live WAF request processing pipeline. The RandomizedSearchCV tuning process takes an additional 10-15 minutes for training, which is only an offline cost and does not impact production inference. Although Logistic Regression is faster than LightGBM, as discussed in Sections 4.2.1 and 4.3.2, respectively, it fails on SQL injection and cannot be used as a general-purpose WAF classifier because of that. The DNN/MLP is the most efficient of the deep learning models. With a latency of 2 to 5 milliseconds per 1000 requests on CPU, it's suitable for most production WAF deployments and, unlike LSTM and TabNet, doesn't need a special GPU for inference, a major deployment benefit over these models as they are more compute-intensive when predicting as well as training. At 10 to 15 milliseconds per 1,000

requests, LSTM's latency is the highest of all the architectures and is not suitable for high throughput production WAF environments that can receive up to thousands of requests per second. The CNN has moderate inference speed but as seen in Table 4.8, has poor recall on SQL injection thus decreasing its value for use as a WAF component.

The proposed stacking ensemble comes with a moderate computational overhead when compared to its base learners, with the inference time of 3-8 milliseconds per 1,000 requests. This overhead is due to the fact that the ensemble requires two consecutive forward passes: a forward pass through XGBoost followed by a forward pass through the MLP, and then a forward pass through Logistic Regression meta-learner with the concatenated probability vector as input. This latency in absolute terms is still within a range acceptable for WAF deployment: The latency of the ensemble is less than 0.01 millisecond per single HTTP request, even at the worst case of 8 milliseconds per 1,000 requests. The training time of 60 to 90 minutes for the entire stacking pipeline (5-fold out-of-fold base learner training, plus meta-learner fitting) is a reasonable one-time investment for initial deployment and periodic retraining of the models (when new attack patterns appear).

The overall verdict is that if it is a single model deployment, XGBoost is the best choice for the best performance-per-compute trade-off, and if the extra overhead of the inference is acceptable, and you need extra robustness for ensemble classification, then you should use the stacking ensemble. Based on the results of performance, neither the LSTM nor Autoencoder seems to be a competitive computational profile, making both unsuitable for production WAF deployment strictly on the basis of computation without taking into account classification limitations.

4.4.4 Model Ranking Summary

The overall ranking of the five most significant models evaluated in this research are given in Table 4.15, which are based on the combined performances of the models in individual data set evaluations and the merged data set configuration. The ranking is determined by the balance between precision and recall, as well as consistency in performance across different types of attacks and reliability of calibration.

Table 4.16 – Overall Model Ranking Summary

Rank	Model	Strongest On	Merged F1	Recommended Context	Deployment
1	Stacking Ensemble (Proposed)	All datasets	0.9592	Multi-attack WAF — balanced precision-recall across all attack types	
2	MLP / DNN (Tuned)	SQL, OSC	0.9671	High-recall WAF — when attack detection rate is paramount	
3	XGBoost (Tuned)	XSS, SQL	0.9591	Production single-model — fastest inference, interpretable	

4	TabNet	LFI, OSC	0.9825*	Interpretable DL — attention-based feature selection
5	SVM (RBF)	SQL, XSS	0.9755*	Lightweight classical — minimal infrastructure requirement

The stacking ensemble is not the top performer on any individual measure (tuned MLP achieves the highest recall and F1-Score for the merged data) but is the most balanced in terms of precision-recall profile, has the highest accuracy, and has the highest ROC-AUC as the best individual model. Although merged-dataset F1 scores place XGBoost third, it comes second when considering production recommendation because of its consistent precision and easy interpretation of feature importances. In specific scenarios, such as those that require model interpretability or lightweight deployment, TabNet and SVM are highlighted as viable alternatives.

4.5 Summary

This chapter introduced a comprehensive performance analysis of eleven machine learning and deep learning models for WAF ID on four attack datasets and on a multi-class configuration of the merged attack datasets. The classical ML evaluation showed that gradient-boosted tree ensembles are the most consistent and balanced solution for all attack types, with near zero overfitting risk and on most datasets no worse performance than other tree-based methods. The one outlier, Logistic Regression's failure at 84.57% in SQL injection, was an important empirical proof that non-linear modelling is a must-have in any production WAF system to achieve a robust SQL injection detection.

In the deep learning track, the DNN (MLP) proved to be the best model for all four attack types, and obtained the highest F1-Score for SQL (0.9902) and XSS (0.998) and decent results for LFI and OSC. The uniform poor recall of the Autoencoder on all the datasets validated the fact that reconstruction-based anomaly detection alone can't be used as a WAF mechanism when labelled training data is available. The Optuna-tuned MLP achieved consistently strong results across all four datasets, with accuracy ranging from 93.84% to 99.52% and ROC-AUC values of 0.95 or above in every case, confirming that Bayesian hyperparameter optimization produces robust, well-generalizing configurations regardless of attack type.

The stacking ensemble model with XGBoost and MLP as base learners and Logistic Regression as the meta-learner showed the best performance with the highest accuracy (96.09%) and balanced precision-recall profile on the combined multi-class data set. The improvement over the individual base learners is not dramatic, but rather is consistent with the near optimal tuning of each of the two classifiers, and the ensemble offers a more robust and consistent classifier than either does on its own; the ensemble is the recommended architecture for production deployment of multi-attack WAF. The next chapter ends the research by discussing limitations, practical implications, and future research directions.

CHAPTER 5 CONCLUSION

5.1 Overview

This thesis aimed at designing, implementing and evaluating an ML-based web application firewall (WAF) to reliably identify four major categories of web injection attacks – Local File Inclusion (LFI), Cross-Site Scripting (XSS), SQL Injection (SQLi) and Operating System Command Injection (OSC) in the same multi-class classification framework. The motivation of this research is the existing lack of signature-based and rule-based WAFs, which are structurally hard to adapt to new attack variants and demand constant manual maintenance, and can be evaded by encoding and obfuscation techniques that maintain the semantic maliciousness of a payload while avoiding pattern-matching rules. This solution is proposed by stacking the ensemble architecture, in which the ensemble model is composed of a gradient-boosted tree model and a deep neural network, with predictions from the two models connected by a logistic regression meta-learner, which is trained on the out-of-fold predicted probabilities.

Eleven potential models were tested in a parallel track experiment. The classical machine learning track evaluated XGBoost, Random Forest, LightGBM, CatBoost, Support Vector Machine with RBF kernel, and Logistic Regression. The deep learning track evaluated Multilayer Perceptron (MLP/DNN), TabNet, LSTM, CNN, and Autoencoder. All models are tested on a hybrid version of the CSIC 2010 and HTTPParams 2015 benchmark data sets, and the class imbalance is handled by SMOTE applied only in the training portion to prevent data leakage, while keeping up with the latest real-time HTTP attack data gathered by automated attack simulations. Tuned versions of the best models of each track (XGBoost and MLP) were built using the RandomizedSearchCV and Optuna Bayesian optimization methods, respectively, and then incorporated into the proposed stacking ensemble. Five complementary metrics were used to quantify the performance: Accuracy, Macro-Precision, Macro-Recall, Macro-F1-Score, and ROC-AUC, which were aided by SHAP feature importance analysis, ROC curve evaluation, and examination of the confusion matrix.

The experimental results showed that both classical and deep learning models work well in detecting structured HTTP request features. Out of all four attack datasets, the XGBoost model was the most consistent, with a tuned configuration that reported classification accuracy of 93.842% on LFI, 99.803% on XSS, 98.638% on OSC, and 99.481% on SQL, and there was negligible overfitting risk. The DNN was the best deep learning model, with the highest F1-Score of 0.9902 and recall of 0.9985 on SQL injection, which is better than XGBoost in recall on SQL injection. Production deployment of the proposed WAF solution with multiple attacks was confirmed on the merged multi-class dataset, where the highest overall accuracy (96.09%)

and most balanced precision-recall profile (Macro-F1-Score of 0.9592 and ROC-AUC of 0.9765) were achieved by the proposed stacking ensemble.

This chapter reviews the research findings and presents a summary of the contributions made in the chapters, discusses the limitations of the research and implications for interpretation of results, and suggests specific future directions that might extend the scope and impact of the proposed system.

5.2 Summary of Research Findings

From the classical machine learning evaluation, several important findings were generated, with direct implications for the design of a WAF system. Four of the six algorithms evaluated were tree-based ensemble methods— XGBoost, Random Forest, LightGBM, and CatBoost—which converged to statistically indistinguishable performance on the LFI, XSS, and OSC datasets, suggesting that any well-configured tree ensemble can achieve the same classification maximum on the engineered HTTP feature space without any further architectural distinction. This convergence is a practically important finding as it indicates that any of the four tree-based methods could be used within the classical component of a WAF system with similar expected performance, and this choice can thus be dictated by other concerns such as inference latency, memory footprint, or feature interpretability instead of classification accuracy.

In the classical evaluation, the worst result was Logistic Regression's collapse on the SQL injection dataset, with only 84.57% accuracy and a Macro-F1-Score of only 0.726, over fifteen percentage points worse than the next-worst classical model on that dataset. Logistic Regression, on the other hand, performed well with all the other types of attacks with over 92% accuracy, suggesting that the failures are not due to a general inability of the model but specific to the feature space of SQL injection. This finding offers strong empirical evidence of the feature interaction effects of SQL injection patterns that are not captured by a linear decision boundary and a direct practical warning: any WAF component using a linear classifier for SQL injection detection would miss about a quarter of SQL injection attempts, resulting in a critical gap in the security layer. This failure mode is not observed in the non-linear models (tree ensembles and neural networks), highlighting the need for non-linear classification for robust protection against SQL injection attacks.

In the deep learning stream, the DNN performed best overall on each of the four data sets, and showed the most significant performance improvement over classical methods on the SQL injection data set with its recall. The Autoencoder's uniformly poor recall score (less than 15% on all datasets) shows that reconstruction-based anomaly detection is fundamentally not suitable to be used in a supervised WAF classification, if labelled training data is available. The model is biased to predict the majority class (benign) whatever the input, which results in

high precision by having to predict nearly all samples as benign and missing almost all attacks. On the LFI dataset, TabNet achieved competitive performance and marginal improvements which may be due to its sequential attention mechanism that seems to have a slight advantage in determining the most discriminative combinations of path traversal tokens. Both sequential temporal modelling (LSTM) and local convolutional feature extraction (CNN) have limitations when applied to SQL injection: LSTM reached only 95.14% accuracy while CNN only reached 85.53% accuracy, both of which were less than the accuracy obtained on XSS and OSC.

The results of the hyperparameter optimization showed that all the tuned XGBoost models perform well across the train and test sets, showing a difference of less than 0.002 on all four datasets, with all differences adhering to the low-risk criterion. The Optuna-optimised MLP achieved consistently strong performance across all four datasets — LFI, XSS, SQL, and OSC — with accuracy between 93.84% and 99.52% and ROC-AUC of 0.95 or higher throughout, with XSS attaining the highest ROC-AUC (0.9985) of the four configurations. This confirms that the Tree-structured Parzen Estimator search in Optuna reliably finds configurations that generalize well across structurally different attack categories. The results on individual datasets confirmed that the ensemble performance in these was very close to that of the best individual base learner, with the ensemble being marginally better on SQL injection (SQLI) where the higher recall of the MLP gives a genuine complementary signal, and exactly the same as the best individual model on LFI and OSC. The ensemble had the best accuracy of 96.09% on the merged hybrid dataset, and the best trade-off between precision and recall among the three final models (here, precision and recall are defined for class level 1, as the class of interest), which evidences that the fusion of precision-oriented tree partitioner XGBoost and recall-oriented neural representation learning MLP, through a well-designed out-of-fold stacking procedure, yields a more stable multi-attack model than either model alone. The improvement is modest, not spectacular, since both base models are already very optimised, but the fact that the ensemble performed better across all the evaluation metrics, accuracy, precision-recall trade-off, and ROC-AUC metric, means that it is the best architecture to deploy in production.

5.3 Research Contributions

This research contributes to the state of the art machine learning based web application firewall design (WAF) in five specific ways to fill the gaps identified in Chapter 2. The first and most fundamental is the building and systematic evaluation of a comprehensive comparative framework, which simultaneously compares eleven candidate models (six classical and five deep-learning models), against all four major categories of injection attacks and against a merged multi-class injection attack dataset. This width had not been systematically compared in a controlled manner in a single experimental setting with the same data handling, preprocessing, and evaluation metrics in any previous study in the WAF intrusion detection literature. This comparative framework gives the web application security research community empirically validated hints at which models are best suited for each type

of WAF deployment, which suggests using: XGBoost as the best classical model, DNN as the best deep learning model for high recall deployments and the stacking ensemble as the best model for multi-attack deployments.

The second contribution is the proposal, and empirical validation of a stacking ensemble architecture with a Logistic Regression meta-learner trained on out-of-fold probability predictions, for WAF IDS combining a gradient-boosted tree model and a deep neural network. Another type of stacking ensembles, which combine heterogeneous base learners from different algorithmic families, had not been investigated with regard to WAF classification. A key design aspect of the implementation of the out-of-fold prediction procedure is that it avoids the leakage of information from the base learner learning step to the meta-learner learning step, which would lead the ensemble to overfit to the predictions on the training set and hence inflate its performance estimates. Chapter 3 provides the theoretical rationale behind this particular choice of base learners: the complementary inductive biases of the gradient boosting and neural network on structured tabular features are explained and the empirical result reported in Chapter 4 confirms this.

The third is using and systematic recording of two hyperparameter optimisation frameworks for WAF classification models: RandomizedSearchCV with 200 iterations and 10-fold cross-validation for XGBoost and Optuna based on Bayesian optimisation with Tree-structured Parzen Estimation for MLP. The hyperparameter values found in these search processes for each dataset and the overfitting gap analysis that validates their generalisation properties represent a reproducible methodology for hyperparameter tuning that can be readily applied to new WAF datasets or new attack type coverage without having to start from default hyperparameter values.

The fourth one is the feature importance analysis using SHAP which has been applied to the base learner (XGBoost) for each of the four attack datasets and the hybrid dataset obtained by merging all the four attack datasets. This analysis goes beyond the overall measures of performance to offer an interpretable, instance-level breakdown of the model's predictions for each attack type and identifies which of the engineered features are important for the classification for each attack type: the frequency of certain keywords, the number of characters, and the structural attributes of requests. The interpretability is beneficial for WAF operators who must know what signals a model is looking for, which could be adversarial bypass, and explain detection rationale to security stakeholders. The SHAP analysis also proves the validity of the feature engineering methodology as the most important features based on the SHAP values equal the known syntactic features of each attack type.

The fifth contribution is the demonstration that Optuna's Tree-structured Parzen Estimator search produces MLP configurations that generalize consistently across four structurally distinct attack categories — achieving accuracy between 93.84% and 99.52% and ROC-AUC above 0.95 in every case (Table 4.11) — without requiring per-dataset architectural redesign.

Notably, the search converged on nearly identical hyperparameter configurations (512-256-128 hidden layers, learning rate ≈ 0.00314 , dropout ≈ 0.185) for LFI, SQL, and OSC, while finding a smaller, distinct configuration for XSS (128-64-32, dropout 0.2186), yet all four configurations achieve near-ceiling discriminative performance. This shows that Bayesian hyperparameter optimization is a reproducible, dataset-agnostic methodology for tuning deep learning components of WAF classifiers.

5.4 Limitations of the Study

However, a few limitations of this research can be stated as follows: 1) The proposed system involves an external UAV and operator, which introduces additional costs, technical complexity and potential time delays. 2) The proposed system is not autonomous and requires additional personnel to control the external UAV.

The basic constraint is the type of input feature representation. The models used in this study are based on a small number of manually created features based on the count of the occurrence of a special character in the fields of an HTTP request and the frequency of occurrence of keywords in a request. It is compact and computationally efficient and in the literature it has been practically validated, but it only reflects lexical and surface syntactic features of the content of the request. It does not maintain sequential order of request tokens or semantic connections between keywords or context of features in the request or application-level knowledge of the usage of request parameters. Therefore, when an attack payload is structurally different from the training data at the feature level, the models can no longer detect such attacks and the LFI performance upper bound of $\sim 93.84\%$ is likely due to an unavoidable Bayes error in the feature space from the overlap of patterns of legitimate paths and attack payloads.

A drawback of this is that the trained models are static. The classifiers developed in this study are trained only once on a fixed set of data, and are then deployed as it is, with no adaptation mechanism to adjust to new attacks arriving after the training cut-off. Real-world web application attacks are continually changing as attackers discover new evasion techniques, exploit new application vulnerabilities and adjust their payloads to work around deployed security countermeasures. A static model trained with the hybrid data set in this work would need to be periodically retrained with new data to remain effective in detecting new attacks, and the time between a new attack variant becoming available and sufficient labelled training data being available for retraining the model would be a constant period of lowered protection. Incremental or online learning strategies are not discussed, which might enable the model to continuously adapt without retraining using a stream of labelled traffic data.

Another restriction is that they can only attack with a certain area. The experimental study covers four categories of injection attacks, covering a significant, but not comprehensive, part

of the web application attack space. The OWASP Top 10 also has other high-risk attack categories that are not covered in the training data: Cross-Site Request Forgery (CSRF), XML External Entity injection (XXE), Server-Side Request Forgery (SSRF), and broken authentication attacks. These attack types would not be protected under a WAF system based on the proposed models, and including them in the framework would involve extra labelled data, feature engineering modifications and maybe changes in architecture to take into account the different syntactic characteristics of these attacks. The four types of attack examined were chosen to correspond to the data available in the published hybrid collection; the extent of representativeness of the trained models is limited by this range.

5.5 Future Work

Based on the limitations discussed above and the patterns and anomalies found in the experimental results, several concrete directions are suggested for future research that can greatly enhance the scope and strength of the proposed WAF system.

The most effective near-term extension would be replacing or augmenting the hand-crafted feature representation with a learned textual representation of content of the HTTP requests. This thesis relied on a frequency-based feature representation for raw HTTP request text, but other types of representations like dense contextual embeddings from pre-trained transformer models, like BERT, could be trained on raw HTTP requests to capture position-sensitive co-occurrences, negations, and syntactic structures that are not being captured by the frequency-based feature representation. A WAF using transformer embeddings could identify new variants of existing attacks that try to make the attack intent obvious while changing the way it is represented on the surface to circumvent keyword frequency detections. The main issue for this extension is the use of a transformer for the inference in a real time request processing pipeline, which would require model distillation, quantization or using lighter transformer architectures that are more suitable for low latency deployment.

Another high priority direction is expanding the attack coverage to the entire attack surface of the OWASP Top 10 injection categories. There are other syntactic signatures for CSRF, SSRF, XEE and RFI, which can be modelled using the same feature engineering and classification framework as that developed in this thesis, assuming that there are large enough and representative labelled databases. If a complete multi-attack dataset for all OWASP injection categories exists (and preferably combines existing benchmark data with new real-time simulations against the latest vulnerable application environments), it will be possible to develop a true general purpose ML-based WAF that covers the entire surface area of injection attacks and not just a subset. This extension would also allow to test the validity of the stacking ensemble architecture with higher number of attack classes.

Given that Optuna converged on nearly identical hyperparameter configurations for three of

the four datasets (LFI, SQL, OSC) while finding a distinct, smaller configuration for XSS, future work could investigate transfer learning or warm-starting strategies — using the hyperparameters or learned weights from one attack type as initialization for tuning on a new attack category — to reduce the computational cost of extending this framework to additional OWASP Top 10 categories.

This is because trained models are static and this pushes for online and incremental learning strategies for WAF ID. The continual learning frameworks would be able to update the parameters from a sequence of new labelled examples that come in as the attacks occur: a security analyst who labels flagged requests as confirmed attacks or false positives would be able to reduce the retraining latency for new attack variants. There are no online learning base learners that can be directly used to update gradient-boosted models; however, neural network base learners can be used via incremental mini-batch gradient descent. A hybrid architecture that continuously updates the MLP base learner and periodically retrains the XGBoost component on the collected data could offer the benefit of adaptability of online learning and stability of batch-trained tree ensembles. Catastrophic forgetting, a well-known problem in neural networks where the performance of the learned tasks degrades when the network is fine-tuned on new tasks, would require regularization methods like replay or elastic weight consolidation.

Adversarial robustness would be a promising direction for future work, in which future evaluation of the proposed ensemble would involve an adversary with partial knowledge of the model who would make a conscious effort to create request payloads that are not detected. The effectiveness of adversarial machine learning (AML) methods such as gradient-based perturbation, genetic algorithm-based payload mutation, and reinforcement learning-based evasion has been shown to be effective against neural network-based classifiers in related security applications, but it remains unclear whether the proposed stacking ensemble is susceptible. The ability of the ensemble to hold up to these attack strategies, as well as the development of defense strategies like adversarial training, input preprocessing to remove common obfuscation, and ensemble diversity regularization to fight coordinated adversarial examples, would make the system more robust and secure in deployment scenarios where sophisticated adversaries are expected. Future work can directly build on the foundation of the evolutionary testing framework proposed by Liu et al., the evolutionary testing framework proposed by Li et al., and the adversarial robustness evaluation proposed in this paper.

Finally, the proposed system should be tested in a real production web environment with real traffic to ensure that the performance estimates collected from an offline benchmarking test are applicable when the system is running in the real operational environment. The train-test separation and overfitting analysis that was employed for this thesis is quite rigorous, and is sufficient to serve the purposes of this thesis, but cannot fully describe the distribution shift between the training set and the traffic pattern of a specific production application, or the latency requirements of a real-time request processing pipeline, or the operational difficulty of managing a false positive rate at a scale of thousands of requests per second. The most

meaningful validation of the practical value of the proposed approach would be to conduct a deployment study that compares real-world false positive burden and missed detection rate between the ML-based WAF and a baseline approach that uses traditional rules.

5.6 Concluding Remarks

The field of web application security is one of the most impactful and dynamic areas of applied cybersecurity research. The economic and social impact of successful injection attacks against web applications is increasing as the web is increasingly becoming the primary means of interaction for digital commerce, healthcare, government, and social interactions. The contributions of this thesis have showed that, machine learning based WAF systems, and the proposed stacking ensemble in particular, provide a credible and technically sound direction to a more adaptive, robust and comprehensive injection attack detection than the traditional rule-based approach can offer.

The experimental approach to each of the five research objectives set out in Chapter 1 has been undertaken in Chapters 3 and 4. A hybrid database for all four types of attacks was built and made available for reusability. The various candidate models were evaluated systematically in a controlled environment, using the same preprocessing and evaluation methods. For each track, an optimal configuration of the two models, XGBoost and MLP, respectively, was obtained through dedicated hyperparameter optimization, thus yielding well-generalized configurations. The stacking ensemble is designed, implemented and validated with proper out-of-fold procedures and five evaluation-metrics, SHAP feature importance analysis, ROC curves and confusion matrices are used to thoroughly characterize the performance. The results show that the ensemble outperform all the architectures tested for multi-attack WAF deployment with the most balanced and robust performance profile.

It's worth noting that the ensemble's performance difference is small and statistically significant (0.01 to 0.02 percentage points in most metrics) over the performance of the independent learners that have remained consistent when the ensemble is tuned to this point. The ensemble's performance improvement, though small, is statistically significant (0.01 – 0.02 percentage points) compared to the base learners in most metrics, and this is itself a meaningful finding. It sets the current state-of-the-art for the feature representation and dataset as a benchmark that is being closely questioned by individual well-optimised models, and it indicates that further significant improvement will come from improvements in feature representation, the size and variety of the dataset and/or model architecture, not simply by ensemble optimisation. The limitations found in this thesis highlight not the shortcomings of the proposed method but the real limitations of data and representations in assisting the future work directions outlined in Section 5.5.

The models learned security-relevant patterns from the training data and not spurious correlations as the results from the SHAP analysis show. The features with the highest

importance values in all four attack categories match exactly with the syntactic signatures of these attacks as described in the security literature, thus confirming the feature engineering approach and the models' classification mechanism. This match between model's learned decision logic and domain expert knowledge is an essential component of trust in any machine learning system used in a security-critical environment, and it is a foundation for the wider adoption of these types of ML-powered WAF systems in production environments, where the explainability and auditability of such systems are regulatory or operational requirements.

To summarize, this study shows that a carefully crafted stacking ensemble of two base learners, XGBoost and MLP, when trained on a modern hybrid dataset that is properly class-balanced and optimized for the hyperparameters, performs at a state-of-the-art level for the four main categories of injection attacks to detect web application firewall abuse on web applications. The experimental setup, architecture proposed, and the analytical method developed in this thesis lay a solid and repeatable groundwork for future research on adaptive, interpretative, and operationally robust machine-learning-enhanced systems to protect web applications.

BIBLIOGRAPHY

- [1] OWASP Foundation, "OWASP Top Ten Web Application Security Risks," 2021. [Online]. Available: <https://owasp.org/www-project-top-ten/> [Accessed: Jun. 2026].
- [2] F. Hock and P. Kortis, "Commercial and open-source based intrusion detection system and intrusion prevention system (IDS/IPS) design for an IP networks," in Proc. 13th Int. Conf. Emerging eLearning Technologies and Applications (ICETA), Stary Smokovec, Slovakia, 2015, pp. 1–4.
- [3] P. Luptk, "Bypassing web application firewalls," in Proc. 6th Int. Scientific Conf. on Security and Protection of Information, 2011, pp. 79–88.
- [4] K. Shipulin, "We need to talk about ids signatures," Network Security, vol. 3, pp. 8–13, 2018.
- [5] B. Isiker and I. Sogukpinar, "Machine learning based web application firewall," in Proc. 2nd Int. Informatics and Software Engineering Conf. (IISEC), Ankara, Turkey, 2021, pp. 1–6.
- [6] A. Shaheed and M. H. D. B. Kurdy, "Web application firewall using machine learning and features engineering," Security and Communication Networks, vol. 2022, pp. 1–14, 2022.
- [7] J. A. Román-Gallego, M. L. Pérez-Delgado, M. L. Viñuela, and M. C. Vega-Hernández, "Artificial intelligence web application firewall for advanced detection of web injection attacks," Expert Systems, vol. 42, no. 1, p. e13505, 2023.
- [8] M. E. Durmuskaya and S. Bayrakli, "Web application firewall based on machine learning models," PeerJ Computer Science, vol. 11, p. e2975, Jul. 2025.
- [9] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," Journal of Artificial Intelligence Research, vol. 16, pp. 321–357, 2002.
- [10] E. Y. Guven, S. Gulgun, C. Manav, B. Bakir, and Z. G. Aydin, "Multiple classification of cyber-attacks using machine learning," Electrica, vol. 22, no. 2, pp. 313–320, 2022.
- [11] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in Proc. 22nd ACM SIGKDD Int. Conf. on Knowledge Discovery and Data Mining, San Francisco, CA, 2016, pp. 785–794.
- [12] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: A next-generation hyperparameter optimization framework," in Proc. 25th ACM SIGKDD Int. Conf. on Knowledge Discovery and Data Mining, 2019, pp. 2623–2631.
- [13] C. T. Giménez, A. P. Villegas, and G. Á. Marañón, "HTTP data set CSIC 2010," Information Security Institute of CSIC, Madrid, Spain, 2010. [Online]. Available: <http://www.isi.csic.es/dataset/>
- [14] Morzeux, "Httpparams," GitHub Repository, 2020. [Online]. Available: <https://github.com/Morzeux/HttpParamsDataset>
- [15] Z. Ghanbari, Y. Rahmani, H. Ghaffarian, and M. H. Ahmadzadegan, "Comparative approach to web application firewalls," in Proc. 2nd Int. Conf. on Knowledge-Based Engineering and Innovation (KBEI), 2015.
- [17] A. H. Yaacob, N. M. Ahmad, N. N. Ahmad, and M. Roslee, "Moving towards positive security model for web application firewall," Int. Scholarly and Scientific Research and Innovation, vol. 6, no. 12, pp. 1763–1768, 2012.

- [18] M. Ismail, S. Alrabae, S. Harous, and K. K. R. Choo, "Empirical evaluations of machine learning effectiveness in detecting web application attacks," in *Future Access Enablers for Ubiquitous and Intelligent Infrastructures*, Cham: Springer, 2023, pp. 99–116.
- [19] DURMUŞKAYA, Muhammed Ersin; BAYRAKLI, Selim, "ML Based WAF-mitmproxy: csv and log files," figshare. Dataset. 2024. <https://doi.org/10.6084/m9.figshare.26583526.v1>
- [20] J. Harish Kumar and J. J. Godwin Ponsam, "Cross site scripting (XSS) vulnerability detection using machine learning and statistical analysis," in *Proc. 2023 Int. Conf. on Computer Communication and Informatics (ICCCI)*, Coimbatore, India, 2023, pp. 1–9.
- [21] S. Kascheev and T. Olenchikova, "The detecting cross-site scripting (XSS) using machine learning methods," in *Proc. 2020 Global Smart Industry Conf. (GloSIC)*, Chelyabinsk, Russia, 2020, pp. 265–270.
- [22] G. Habibi and N. Surantha, "XSS attack detection with machine learning and n-Gram methods," in *Proc. 2020 Int. Conf. on Information Management and Technology (ICIMTech)*, Bandung, Indonesia, 2020, pp. 516–520.
- [23] A. K. Kassem, "Intelligent system using machine learning techniques for security assessment and cyber intrusion detection," Ph.D. dissertation, Université d'Angers, France, 2021.
- [24] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T. Liu, "LightGBM: A highly efficient gradient boosting decision tree," in *Advances in Neural Information Processing Systems (NIPS)*, vol. 30, 2017, pp. 3146–3154.
- [25] L. Prokhorenkova, G. Gusev, A. Vorobev, A. V. Dorogush, and A. Gulin, "CatBoost: Unbiased boosting with categorical features," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2018, pp. 6638–6648.
- [26] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [27] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [28] S. Ö. Arik and T. Pfister, "TabNet: Attentive interpretable tabular learning," in *Proc. AAAI Conf. on Artificial Intelligence*, vol. 35, no. 8, 2021, pp. 6679–6687.
- [29] D. H. Wolpert, "Stacked generalization," *Neural Networks*, vol. 5, no. 2, pp. 241–259, 1992.
- [30] B. R. Dawadi, B. Adhikari, and D. K. Srivastava, "Deep learning technique-enabled web application firewall for the detection of web attacks," *Sensors*, vol. 23, no. 4, p. 2073, 2023.
- [31] L. K. Shar, L. C. Briand, and H. B. K. Tan, "Web application vulnerability prediction using hybrid program analysis and machine learning," *IEEE Transactions on Dependable and Secure Computing*, vol. 12, no. 6, pp. 688–707, 2015.
- [32] Y. Tao and K. Meng, "URL based file inclusion attack behavior analysis and an autoencoder detection model," in *Proc. 8th Int. Conf. on Computing and Artificial Intelligence*, 2022, pp. 252–258.
- [33] M. Liu, K. Li, and T. Chen, "Security testing of web applications: a search-based approach for detecting SQL injection vulnerabilities," in *GECCO 2019 Companion*, 2019, pp. 417–418.
- [34] K. Li, H. Yang, and W. Visser, "DaNuoYi: evolutionary multi-task injection testing on web application firewalls," *IEEE Transactions on Software Engineering*, 2023.